

ForeSyte

Project Team

Inam Ullah Shaikh	22i-0857
Emaan Fatima	22i-0869
Aden Sial	22i-1313

Session 2022-2026

Supervised by

Ms. Saira Qamar



Department of Computer Science

**National University of Computer and Emerging Sciences
Islamabad, Pakistan**

June, 2026

Contents

1	Introduction [AS PER SCOPE DOCUMENT]	1
1.1	Existing Solutions	1
1.1.1	How ForeSyte is Different	2
1.2	Problem Statement	3
1.3	Scope	4
1.4	Modules	6
1.4.1	ForeSyte Web Portal	6
1.4.2	Backend Service	6
1.5	Work Division	7
2	Project Requirements [AS PER FYP1 MID REPORT]	9
2.1	Use-cases	9
2.2	Functional Requirements	17
2.3	Non-Functional Requirements	22
2.3.1	Reliability	23
2.3.2	Usability	23
2.3.3	Performance	23
2.3.4	Security	23
2.3.5	Maintainability	24
3	System Overview [AS PER FYP1 MID REPORT]	25
3.1	Architectural Design	25
3.2	Domain Model	29
3.3	Design Models [UPTO THE CURRENT ITERATION ONLY]	30
3.3.1	Class Diagram	30
3.3.2	Activity Diagram	31
3.3.3	State Transition Diagram	39
4	Implementation and Testing	41
4.1	Current Progress	41
4.1.1	Front-End	41
4.1.2	Back-End	42
4.1.3	Next Steps	44

4.2	Algorithm Design	44
4.2.1	Admin Dashboard Aggregation	45
4.2.2	Report Generation Workflow (Planned Design)	46
4.2.3	Investigator Violation Review Flow	46
4.2.4	Seating Plan Upload and Processing	47
4.3	External APIs and Libraries	51
4.4	Testing Details	51
4.4.1	Unit Testing	51
References		61
A Appendices		62
A.1	Appendix A	62
A.1.1	Use Case Diagram example (Online Shopping System)	62
A.1.2	Detail Use Case Example	63
A.1.3	Event-Response Table for a Highway Intersection	64
A.1.4	Story Board Example For Android App	65
A.2	Appendix B	66
A.2.1	Domain Model Example For Online Shopping	66
A.3	Appendix C	67
A.3.1	Box And Line Example For Online Shopping	67
A.3.2	Architecture Pattern Example For Online Shopping	68
A.4	Appendix D	69
A.4.1	Activity Diagram	69
A.4.2	Class Diagram	70
A.4.3	Sequence Diagram	71
A.4.4	State Transition Diagram	71
A.4.5	Data Flow Diagram	72

List of Figures

1.1	Estimated reduction in cheating rates using ForeSyte AI-based surveillance system	4
1.2	Frequency of students who reported cheating during written examinations [3]	4
1.3	Scope workflow of the ForeSyte system	5
2.1	ForeSyte Use Case Diagram	17
3.1	Box and Line Diagram for ForeSyte AI Exam Surveillance System	26
3.2	High-Level Architectural Design of the ForeSyte AI Exam Surveillance System	27
3.3	Data Flow and Storage View (PostgreSQL, MongoDB, Blob/Object Storage)	28
3.4	ForeSyte Domain Model Diagram	29
3.5	ForeSyte Class Diagram	30
3.6	ForeSyte Activity Diagram	31
3.7	Sequence Diagram for UC-01: View and Manage Reports/Dashboard . . .	32
3.8	Sequence Diagram for UC-02: Manage User Profiles	33
3.9	Sequence Diagram for UC-03: Generate Detection Reports	34
3.10	Sequence Diagram for UC-04: Verify Student Identity & Seat Mapping .	35
3.11	Sequence Diagram for UC-05: Monitor Invigilator Activity	36
3.12	Sequence Diagram for UC-06: Monitor Student Behaviour	37
3.13	Sequence Diagram for UC-07: Process Exam Footage (Live/Recorded) . .	38
3.14	State Transition Diagram	39
A.1	Use Case Diagram for the Online Shopping System	62
A.2	Detail Use Case Example	63
A.3	Example of Event Response Table	64
A.4	Example of Story Board	65
A.5	Domain Model Example For Online Shopping Application	66
A.6	Box and Line Diagram For Online Shopping Application	67
A.7	Architecture Pattern For Online Shopping Application	68
A.8	Activity Diagram For Online Shopping Application	69
A.9	Class Diagram For Online Shopping Application	70
A.10	Sequence Diagram For Online Shopping Application	71

A.11 State Transition Diagram For Online Shopping Application	71
---	----

List of Tables

1.1	Comparison of Existing Solutions	2
1.2	Work Division among Team Members	8
2.1	Summary of Use Cases for ForeSyte	9
2.2	Use Case 1: View & Manage Reports/Dashboard	10
2.3	Use Case 2: Manage User Profiles	11
2.4	Use Case 3: Generate Detection Reports	12
2.5	Use Case 4: Verify Student Identity & Seat Mapping	13
2.6	Use Case 5: Monitor Invigilator Activity	14
2.7	Use Case 6: Monitor Student Behaviour	15
2.8	Use Case 7: Process Exam Footage (Live/Recorded)	16
2.9	Functional Requirements for UC-01	18
2.10	Functional Requirements for UC-02	18
2.11	Functional Requirements for UC-03	18
2.12	Functional Requirements for UC-04	19
2.13	Functional Requirements for UC-05	19
2.14	Functional Requirements for UC-06	19
2.15	Functional Requirements for UC-07	20
2.16	General Functional Requirements	20
4.1	Unit Test Cases for Reports API	52
4.2	Unit Test Cases for Dashboard APIs	54
4.3	Unit Test Cases for User Management API	56
4.4	Unit Test Cases for Seating Plan Upload	58
4.5	Unit Test Cases for Monitoring and Camera Integration	60

Chapter 1

Introduction [AS PER SCOPE DOCUMENT]

This project proposal presents **ForeSyte**, an AI-powered CCTV/camera monitoring system designed to detect and log cheating attempts and monitor invigilator activity during examinations. The purpose of this document is to outline the problem, motivation, proposed solution, and stakeholders while providing a detailed background of the system.

ForeSyte integrates computer vision and deep learning techniques to analyze both real-time CCTV/camera feeds and uploaded exam footage. By focusing on seat-specific mapping, the system ensures precise tracking of students and their activities while simultaneously monitoring invigilators' behavior to maintain a fair and transparent examination environment [3]. It also ensures correct student ID verification on assigned seats using face recognition, thereby preventing impersonation.

Furthermore, several studies have highlighted the growing prevalence of academic dishonesty and the limitations of traditional manual invigilation methods. By leveraging AI-driven video analysis, **ForeSyte** aims to improve the accuracy of cheating detection, automate timestamped reporting, and enhance overall exam integrity. The system also acts as a deterrent; as students become aware that their activities are continuously monitored, the likelihood of cheating is expected to decrease significantly over time [4].

1.1 Existing Solutions

Several solutions have been developed to address the problem of cheating detection and exam monitoring, each with distinct approaches and limitations. Below is a critical overview and comparison of the most relevant systems:

- **IndoAI:** Employs proprietary CNN-based gesture and face models running on Edge

AI cameras. While it offers AI-driven gesture and face analysis, it does not support seat-based activity detection or invigilator monitoring. Additionally, it lacks the capability to generate structured, timestamped reports. Its reliance on specialized hardware (Edge AI cameras) limits scalability and integration with existing CCTV systems.

- **IPB University System:** Uses dual webcams (front and rear) for each student, combined with AI software for face and gaze monitoring. This approach is not based on CCTV and does not provide seat-based activity detection or invigilator monitoring. While it offers some level of timestamped reporting, it lacks structured, per-seat violation logs and is less scalable for large exam halls due to the need for individual webcams per student.

Table 1.1: Comparison of Existing Solutions

System Name	Technology	Limitations
IndoAI	Proprietary CNN-based gesture + face models on Edge AI cameras.	No seat-based activity detection; no invigilator monitoring; no structured timestamped reports; requires specialized hardware.
IPB University System	Dual webcam (front + rear) per student; AI software (face/gaze monitoring models); not CCTV.	No seat-based activity detection; no invigilator monitoring; limited but no structured per-seat timestamped reports; not scalable for large halls.

1.1.1 How ForeSyte is Different

ForeSyte stands out from existing solutions by addressing their key limitations:

- **Seat-based Activity Detection:** ForeSyte uniquely maps detected activities to specific seats, ensuring identification of students involved in suspicious behavior.
- **Integration with Existing CCTV:** Unlike other solutions that require specialized hardware or individual webcams, ForeSyte works seamlessly with standard CCTV infrastructure, making it scalable and cost-effective for large exam halls.
- **Invigilator Monitoring:** ForeSyte not only monitors students but also tracks invigilator presence and activity, identifying negligence or prolonged absences.

- **Structured, Timestamped Reports:** The system generates detailed, per-seat violation logs with timestamps, enabling efficient review and transparent reporting.
- **Comprehensive Automation:** By combining real-time and recorded video analysis, ForeSyte automates the entire monitoring and reporting process, reducing manual effort and enhancing exam integrity.

This approach ensures that ForeSyte provides a comprehensive, scalable, and automated exam monitoring solution that overcomes the major shortcomings of existing systems.

1.2 Problem Statement

Academic dishonesty has become a significant concern in educational institutions worldwide. Studies indicate that approximately 61% of university students have admitted to cheating during written examinations, while nearly 68% confess to cheating at least once during their academic career [3]. Traditional manual invigilation methods are often insufficient, as invigilators may unintentionally overlook suspicious activities, and reviewing long hours of CCTV/camera footage after exams is both time-consuming and ineffective.

To address these challenges, **ForeSyte** provides an AI-driven solution for automated examination monitoring. The system processes both real-time CCTV/camera streams and uploaded video footage, detecting suspicious activities such as phone usage, hand waving, bending over the desk, looking around, and standing up using advanced computer vision techniques.

The system focuses on seat-specific mapping, ensuring that every detected activity is linked to the correct student. It also ensures that the correct student ID is sitting on the assigned seat by using face recognition. In addition, **ForeSyte** incorporates an Invigilator Activity Monitoring module to:

- Track invigilators' movements
- Detect prolonged idle periods
- Identify potential negligence

An admin dashboard provides real-time monitoring and automated report generation, enabling discipline committees to directly access timestamped violations instead of manually searching through hours of footage. By combining AI-powered surveillance with intelligent analytics, **ForeSyte** enhances exam security and aims to reduce academic dishonesty over time by providing actionable insights. Furthermore, as students become aware that their activities are continuously monitored and every suspicious action is captured with

timestamped reports, the likelihood of cheating is expected to decrease significantly over repeated examination sessions

Figure 1.1: Estimated reduction in cheating rates using ForeSyte AI-based surveillance system

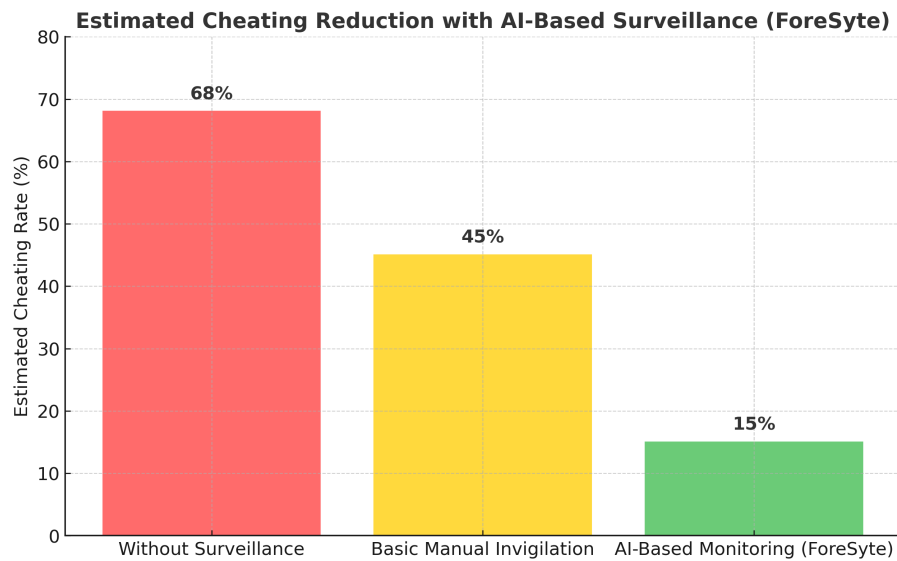
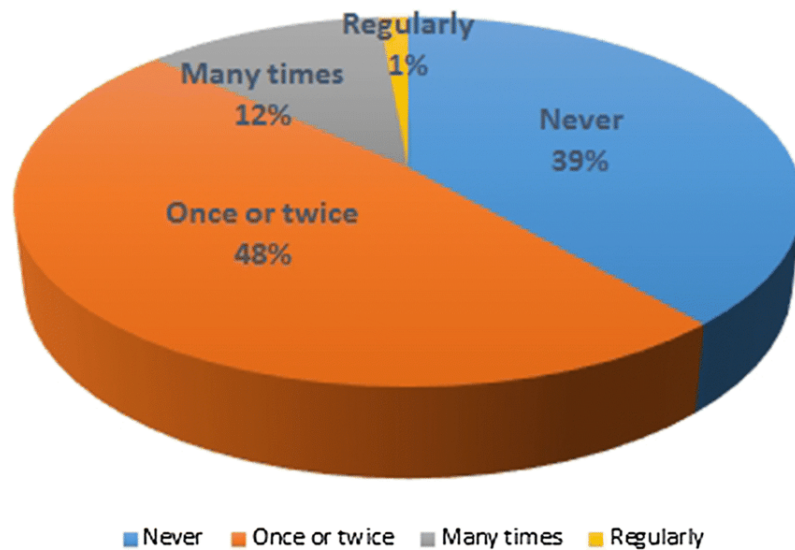


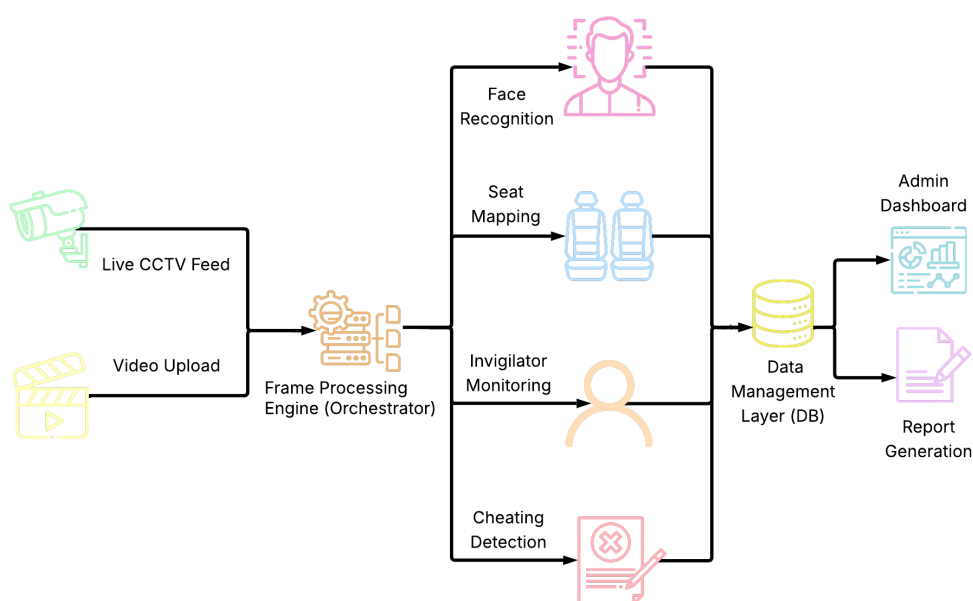
Figure 1.2: Frequency of students who reported cheating during written examinations [3]



1.3 Scope

The scope of the ForeSyte project includes developing an AI-powered exam surveillance system that enhances the integrity of examinations conducted in physical exam halls. Recent research highlights the importance of automated invigilation for maintaining academic integrity [3, 4].

Figure 1.3: Scope workflow of the ForeSyte system



The system will process CCTV/camera footage, both live streams and uploaded recordings, to automatically detect and log suspicious activities such as unauthorized object usage and irregular gestures. It will also incorporate seating arrangement verification, ensuring that each detected activity is accurately linked to the correct student, and that the correct student ID is sitting in the assigned seat using face recognition, similar to approaches in modern proctoring systems [1, 5].

The project further extends to monitoring invigilator activity within the exam hall, identifying negligence or prolonged absence from duty. Similar works on offline and intelligent monitoring systems support this approach [6, 7]. All detected events will be recorded with precise timestamps, seat numbers and room identifiers, enabling a structured and transparent record of violations.

An administrative dashboard will be developed to centralize monitoring and provide clear violation reports supported with visual evidence. This will reduce the need for institutions to manually review extensive CCTV/camera footage, offering instead a concise and accurate summary of flagged incidents.

The system will be designed for real-time performance using optimized frame sampling and AI models capable of efficient video analysis. The project will be self-funded and developed entirely by our team. Related research also discusses the ethical implications of AI in exam supervision, highlighting fairness and privacy issues.

The boundaries of this project are confined to creating a robust and intelligent surveillance solution that automates detection, verification, and reporting within physical exam environments, thereby ensuring greater fairness, efficiency, and accountability in academic assessments.

1.4 Modules

The **ForeSyte** project is organized into several core modules, each responsible for specific functionalities required to ensure seamless, automated exam monitoring. The modules are divided into two main components: the *ForeSyte Web Portal* for stakeholders and the *Backend Service* that powers the AI-driven monitoring capabilities.

1.4.1 ForeSyte Web Portal

The **ForeSyte Web Portal** is a multi-user interface designed for administrators, discipline committee members, and university management. It provides centralized access to seating plans and violation reports, ensuring transparent and efficient management of examination activities.

1. **Dashboard Visualization:** Displays real-time updates of suspicious activities, flagged violations, and invigilator presence across multiple exam halls using data from connected CCTV/camera systems.
2. **Report Access:** Enables authorized users to view, manage, and download detailed violation reports containing timestamps, seat details and room information.
3. **User Roles and Permissions:** Implements secure role-based access, where administrators manage seating plans, camera configurations, and exam schedules, while discipline committee members focus on reviewing reports and initiating actions.
4. **Seating Plan Integration:** Links violations to student seating layouts and verifies correct student IDs using face recognition to ensure accurate identification of candidates involved in flagged incidents.

1.4.2 Backend Service

The **Backend Service** powers the core intelligence of the ForeSyte system, managing video analysis, behavioral detection, invigilator monitoring, and reporting. This module processes video streams from CCTV/camera systems and uploaded exam footage to ensure accurate real-time and recorded exam monitoring.

1. **Video Stream Processing:** Integrates with CCTV/camera systems and uploaded recordings, extracting frames for real-time detection of activities and events.
2. **Cheating Detection Engine:** Uses advanced computer vision techniques to identify suspicious behaviors such as unauthorized object usage, irregular gestures, communication, and frequent seat movements [1, 2].

3. **Face and Seat Verification:** Verifies student identities and ensures correct student ID is sitting on the assigned seat using face recognition, mapping detected activities to seating plans for accurate violation reporting.
4. **Invigilator Monitoring:** Tracks invigilator's prolonged absences from assigned zones, and identifies potential negligence or unauthorized phone usage[6].
5. **Report Generation:** Compiles timestamped violation logs, including room details, seat numbers, and integrates them directly into the ForeSyte Web Portal for automated PDF/Excel report exports.

1.5 Work Division

For each module and respective feature, responsibility is assigned to a team member.

Name	Responsibility
Inam Ullah Shaikh	• Frontend Development • Model Training • Documentation • Deployment • Backend Integration
Emaan Fatima	• Backend Development • Model Research • Model Training • Documentation • Deployment
Aden Sial	• Data Collection • Model Training • Model Research • Documentation • Deployment

Table 1.2: Work Division among Team Members

Chapter 2

Project Requirements [AS PER FYP1 MID REPORT]

This chapter describes the functional and non-functional requirements of the project.

2.1 Use-cases

The Table 2.1 lists down all the possible use cases for our project. Note that login, sign up and feedback pages are not included in these use cases.

Table 2.1: Summary of Use Cases for ForeSyte

Usecase ID	Use Case Name	Description
UC-01	View & Manage Reports/Dashboard	Authorized users view and manage reports of student and invigilator activity.
UC-02	Manage User Profiles	System administrator manages accounts and roles of all users.
UC-03	Generate Detection Reports	System generates timestamped reports of detected cheating or negligence.
UC-04	Verify Student Identity & Seat Mapping	System verifies student identity using face recognition and assigned seats.
UC-05	Monitor Invigilator Activity	System monitors invigilators for negligence, absence, or unauthorized actions.
UC-06	Monitor Student Behaviour	System monitors student actions to detect cheating attempts.
UC-07	Process Exam Footage (Live/Recorded)	System processes live and recorded video streams to detect activities.

Below, you can find the detailed use case description for each use case.

2.1.1 UC-01

Table 2.2: Use Case 1: View & Manage Reports/Dashboard

Use Case Detail	Description
Use Case ID	UC-01
Use Case Name	View & Manage Reports/Dashboard
Scope	ForeSyte Web Portal
Level	User Goal
Brief Description	Users view, filter, and manage real-time dashboards and past reports.
Actor(s) Involved	System Administrator, Investigator, Discipline Committee
Stakeholders	• Discipline Committee: Reviews violation data • Admins: Configure reports
Pre-conditions	Reports and logs already generated by detection system.
Post-conditions	Users access detailed dashboards with downloadable reports.
Main Success Scenario	1. Actor logs into the ForeSyte Web Portal successfully. 2. Actor navigates to the Reports/Dashboard section from the main menu. 3. System retrieves all available violation reports from the database. 4. System displays a dashboard with filters (e.g., by student ID, seat number, violation type, or timestamp). 5. Actor applies filters to refine the report view. 6. System dynamically updates the dashboard according to the selected filters. 7. Actor selects a specific report for detailed view. 8. System displays complete details of the selected report (student, invigilator, time, violation evidence). 9. Actor chooses the option to download or export the report. 10. System generates the report in the selected format (PDF/Excel) and confirms successful export.
Special Requirements	Role-based access, secure data visualization

2.1.2 UC-02

Table 2.3: Use Case 2: Manage User Profiles

Use Case Detail	Description
Use Case ID	UC-02
Use Case Name	Manage User Profiles
Scope	ForeSyte Web Portal
Level	User Goal
Brief Description	System administrator manages user accounts, roles, and permissions.
Actor(s) Involved	System Administrator
Stakeholders	• Admins: Maintain structured access
Pre-conditions	Admin is authenticated.
Post-conditions	User accounts created, modified, or deleted.
Main Success Scenario	1. Admin logs into the ForeSyte Web Portal. 2. Admin navigates to the User Management module. 3. System displays the list of existing users with their roles. 4. Admin selects an action (add, edit, or delete a user). 5. If adding: Admin enters new user details including username, role, and credentials. 6. If editing: Admin updates user details or permissions. 7. If deleting: Admin selects a user and confirms deletion. 8. System validates the request and updates the database accordingly. 9. System confirms success with a message (user added/updated/deleted). 10. Admin logs out or continues with further management tasks.
Special Requirements	Secure password policies, role-based restrictions

2.1.3 UC-03

Table 2.4: Use Case 3: Generate Detection Reports

Use Case Detail	Description
Use Case ID	UC-03
Use Case Name	Generate Detection Reports
Scope	Backend Service + Web Portal
Level	User Goal
Brief Description	System generates timestamped detection reports of suspicious activity.
Actor(s) Involved	Investigator, Discipline Committee
Stakeholders	• Discipline Committee: Reviews reports • University Management
Pre-conditions	Detection system has analyzed video footage.
Post-conditions	Report generated and stored in database.
Main Success Scenario	1. AI detection engine processes exam footage. 2. System identifies suspicious activity (e.g., phone usage, looking around, invigilator negligence). 3. System logs details including timestamp, seat number, room, and involved actor. 4. System compiles the violation data into a structured report format. 5. Investigator accesses the Web Portal and navigates to the Reports section. 6. System displays the list of generated reports. 7. Investigator selects a specific report for review. 8. System displays detailed evidence including video snippets or screenshots. 9. Investigator verifies and approves the report for further review. 10. Report is stored in the system and available for export (PDF/Excel).
Special Requirements	Reports exportable to PDF/Excel

2.1.4 UC-04

Table 2.5: Use Case 4: Verify Student Identity & Seat Mapping

Use Case Detail	Description
Use Case ID	UC-04
Use Case Name	Verify Student Identity & Seat Mapping
Scope	Backend Service
Level	User Goal
Brief Description	System verifies student identity using face recognition and seat mapping.
Actor(s) Involved	Student, Invigilator
Stakeholders	• University Management
Pre-conditions	Seating plan and student ID data available.
Post-conditions	Student identity verified and mapped.
Main Success Scenario	1. Exam seating plan is uploaded into the system. 2. Student enters the exam hall and sits on the assigned seat. 3. System activates facial recognition through CCTV or camera feed. 4. System captures the student's face and compares it with pre-stored student records. 5. System checks whether the student is seated in the correct assigned seat. 6. If the identity matches, the system confirms successful verification. 7. If there is a mismatch, the system flags the seat for manual review by the examiner. 8. System logs the verification results along with seat ID and student ID. 9. Verified data is linked to subsequent activity monitoring and reports.
Special Requirements	Reliable face recognition with low error rate

2.1.5 UC-05

Table 2.6: Use Case 5: Monitor Invigilator Activity

Use Case Detail	Description
Use Case ID	UC-05
Use Case Name	Monitor Invigilator Activity
Scope	Backend Service
Level	User Goal
Brief Description	System tracks invigilators for negligence, absence, or unauthorized actions.
Actor(s) Involved	Invigilator, Discipline Committee
Stakeholders	• University Management
Pre-conditions	Cameras active and invigilator zones defined.
Post-conditions	Invigilator activity log generated.
Main Success Scenario	1. Surveillance cameras start recording invigilator activity. 2. System tracks invigilator's presence across different zones of the exam hall. 3. AI analyzes the video feed to detect movement or prolonged inactivity. 4. If invigilator leaves their zone for too long, the system flags negligence. 5. If invigilator is using unauthorized devices (e.g., phone), system captures it. 6. System logs the timestamp, location, and detected action. 7. Discipline Committee accesses the dashboard to view invigilator logs. 8. System displays flagged negligence cases in the report. 9. Committee reviews and validates the report. 10. Finalized invigilator activity log is stored and exported if needed.
Special Requirements	Accurate zone mapping

2.1.6 UC-06

Table 2.7: Use Case 6: Monitor Student Behaviour

Use Case Detail	Description
Use Case ID	UC-06
Use Case Name	Monitor Student Behaviour
Scope	Backend Service
Level	User Goal
Brief Description	Detects cheating behaviors (phone use, gestures, looking around).
Actor(s) Involved	Student, Investigator
Stakeholders	• Discipline Committee • University Management
Pre-conditions	Camera feed and detection models active.
Post-conditions	Violations logged with timestamps.
Main Success Scenario	1. Exam begins and surveillance cameras monitor all students. 2. AI detection model continuously analyzes student actions. 3. System identifies suspicious behavior such as phone use, hand gestures, or looking around repeatedly. 4. When detected, system flags the behavior as a potential violation. 5. System records timestamp, seat ID, and video evidence. 6. Investigator accesses the dashboard to review logged violations. 7. System displays detailed violation logs with associated video frames. 8. Investigator examines the evidence for each flagged action. 9. Investigator confirms or dismisses the violation. 10. Validated violations are added to the final exam report.
Special Requirements	Must differentiate normal vs cheating actions

2.1.7 UC-07

Table 2.8: Use Case 7: Process Exam Footage (Live/Recorded)

Use Case Detail	Description
Use Case ID	UC-07
Use Case Name	Process Exam Footage (Live/Recorded)
Scope	Backend Service
Level	User Goal
Brief Description	System processes live and recorded exam video footage for analysis.
Actor(s) Involved	System Administrator, Investigator
Stakeholders	• University Management
Pre-conditions	Camera streams or uploaded recordings available.
Post-conditions	Footage analyzed and results stored.
Main Success Scenario	1. System connects to live CCTV cameras or receives uploaded exam recordings. 2. System validates video input and prepares it for analysis. 3. AI engine processes video frames in real time or batch mode. 4. System identifies and logs suspicious student or invigilator behavior. 5. System maps detected activity to the correct student seat. 6. All activities are logged with timestamps and stored in the database. 7. Investigator accesses the Reports module. 8. System displays results of processed footage (violations, seat mapping). 9. Investigator reviews the logged activities and evidence. 10. Reports are finalized and exported in required format.
Special Requirements	Efficient processing, storage optimization

2.1.8 Use Case Diagram of ForeSyte

The Figure 2.1 highlights the use case diagram of the entire ForeSyte Web Application.

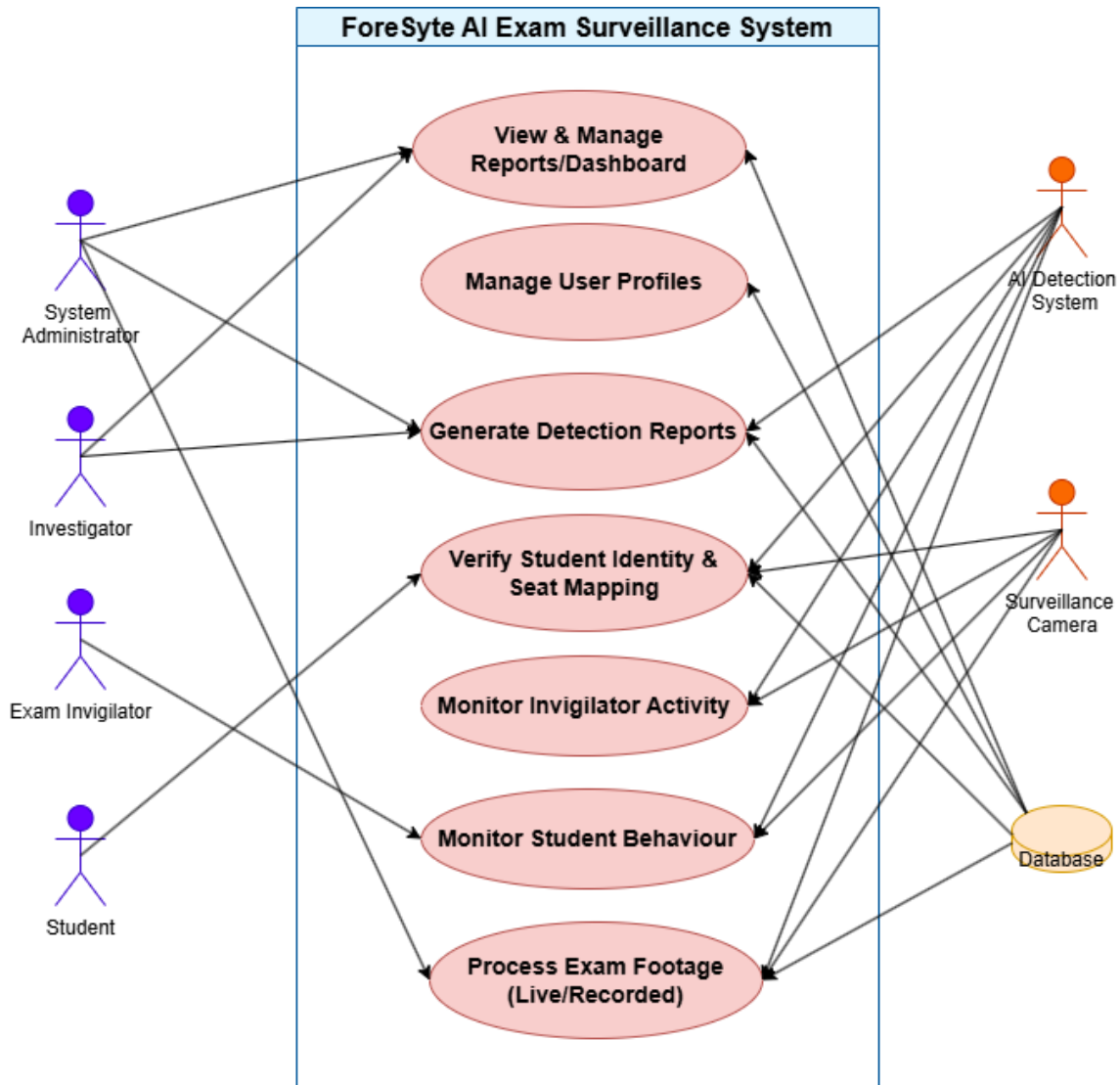


Figure 2.1: ForeSyte Use Case Diagram

2.2 Functional Requirements

This section describes the functional requirements of ForeSyte arranged in a use case-based organizational method.

2.2.1 UC-01 Functional Requirement

Table 2.9: Functional Requirements for UC-01

Functional Requirement ID	Description
FR-01	The system shall allow authorized users to log in and access the Reports/Dashboard module.
FR-02	The system shall display real-time and historical reports of student and invigilator activity.
FR-03	The system shall provide filtering options (by student ID, seat number, violation type, timestamp).
FR-04	The system shall allow exporting reports in PDF and Excel formats.
FR-05	The system shall enforce role-based access to reports.

2.2.2 UC-02 Functional Requirement

Table 2.10: Functional Requirements for UC-02

Functional Requirement ID	Description
FR-06	The system shall allow the administrator to add new user accounts with role assignments.
FR-07	The system shall allow editing of existing user details and permissions.
FR-08	The system shall allow deleting user accounts securely.
FR-09	The system shall validate inputs (username, password, role) before saving.
FR-10	The system shall enforce strong password and security policies.

2.2.3 UC-03 Functional Requirement

Table 2.11: Functional Requirements for UC-03

Functional Requirement ID	Description
FR-11	The system shall automatically generate detection reports when violations are flagged.
FR-12	The system shall include timestamps, seat numbers, and violation type in reports.
FR-13	The system shall store generated reports in a database.
FR-14	The system shall allow investigators to review and approve reports.
FR-15	The system shall allow reports to be exported in PDF/Excel format.

2.2.4 UC-04 Functional Requirement

Table 2.12: Functional Requirements for UC-04

Functional Requirement ID	Description
FR-16	The system shall verify student identity using face recognition.
FR-17	The system shall cross-check student identity against pre-stored records.
FR-18	The system shall map verified students to their assigned seat numbers.
FR-19	The system shall log verification results (student ID, seat ID, verification status).
FR-20	The system shall flag mismatches for manual review.

2.2.5 UC-05 Functional Requirement

Table 2.13: Functional Requirements for UC-05

Functional Requirement ID	Description
FR-21	The system shall monitor invigilators during exam sessions.
FR-22	The system shall track invigilator presence in defined zones.
FR-23	The system shall detect unauthorized activities (e.g., phone use).
FR-24	The system shall log negligence events with timestamp and location.
FR-25	The system shall provide invigilator activity logs to the Discipline Committee.

2.2.6 UC-06 Functional Requirement

Table 2.14: Functional Requirements for UC-06

Functional Requirement ID	Description
FR-26	The system shall analyze student actions in real time.
FR-27	The system shall detect suspicious behaviors (phone use, hand gestures, looking around).
FR-28	The system shall differentiate between normal and abnormal behaviors.
FR-29	The system shall log violations with seat ID, timestamp, and video snippet.
FR-30	The system shall allow investigators to review and confirm logged violations.

2.2.7 UC-07 Functional Requirement

Table 2.15: Functional Requirements for UC-07

Functional Requirement ID	Description
FR-31	The system shall process both live CCTV feeds and uploaded recordings.
FR-32	The system shall analyze frames using AI detection models.
FR-33	The system shall log activities with timestamps and seat IDs.
FR-34	The system shall store processed results in the database.
FR-35	The system shall make analyzed results available in the Reports module.

2.2.8 General Functional Requirements

Table 2.16: General Functional Requirements

Functional Requirement ID	Description
FR-36	The system shall ensure processing of violations and reports within 20 seconds.
FR-37	The system shall implement secure storage and encryption of student/exam data.
FR-38	The system shall allow a responsive web interface for investigators and administrators.
FR-39	The system shall maintain audit logs for all activities.
FR-40	The system shall support scalability for multiple exam halls simultaneously.

2.2.9 Module 1: ForeSyte Web Portal

Following are the requirements for Module 1 (ForeSyte Web Portal):

1. Dashboard Visualization

- (a) FR-01: The system shall allow authorized users to log in and access the Reports/Dashboard module.
- (b) FR-02: The system shall display real-time and historical reports of student and invigilator activity.
- (c) FR-03: The system shall provide filtering options (by student ID, seat number, violation type, timestamp).
- (d) FR-04: The system shall allow exporting reports in PDF and Excel formats.

- (e) FR-05: The system shall enforce role-based access to reports.

2. Report Access

- (a) FR-11: The system shall automatically generate detection reports when violations are flagged.
- (b) FR-12: The system shall include timestamps, seat numbers, and violation type in reports.
- (c) FR-13: The system shall store generated reports in a database.
- (d) FR-14: The system shall allow investigators to review and approve reports.
- (e) FR-15: The system shall allow reports to be exported in PDF/Excel format.

3. User Roles and Permissions

- (a) FR-06: The system shall allow the administrator to add new user accounts with role assignments.
- (b) FR-07: The system shall allow editing of existing user details and permissions.
- (c) FR-08: The system shall allow deleting user accounts securely.
- (d) FR-09: The system shall validate inputs (username, password, role) before saving.
- (e) FR-10: The system shall enforce strong password and security policies.

2.2.10 Module 2: Backend Service

Following are the requirements for Module 2 (Backend Service):

1. Video Stream Processing

- (a) FR-31: The system shall process both live CCTV feeds and uploaded recordings.
- (b) FR-32: The system shall analyze frames using AI detection models.
- (c) FR-33: The system shall log activities with timestamps and seat IDs.
- (d) FR-34: The system shall store processed results in the database.
- (e) FR-35: The system shall make analyzed results available in the Reports module.

2. Cheating Detection Engine

- (a) FR-26: The system shall analyze student actions in real time.
- (b) FR-27: The system shall detect suspicious behaviors (phone use, hand gestures, looking around).

- (c) FR-28: The system shall differentiate between normal and abnormal behaviors.
- (d) FR-29: The system shall log violations with seat ID, timestamp, and video snippet.
- (e) FR-30: The system shall allow investigators to review and confirm logged violations.

3. Face and Seat Verification

- (a) FR-16: The system shall verify student identity using face recognition.
- (b) FR-17: The system shall cross-check student identity against pre-stored records.
- (c) FR-18: The system shall map verified students to their assigned seat numbers.
- (d) FR-19: The system shall log verification results (student ID, seat ID, verification status).
- (e) FR-20: The system shall flag mismatches for manual review.

4. Invigilator Monitoring

- (a) FR-21: The system shall monitor invigilators during exam sessions.
- (b) FR-22: The system shall track invigilator presence in defined zones.
- (c) FR-23: The system shall detect unauthorized activities (e.g., phone use).
- (d) FR-24: The system shall log negligence events with timestamp and location.
- (e) FR-25: The system shall provide invigilator activity logs to the Discipline Committee.

5. Report Generation and Security

- (a) FR-36: The system shall ensure processing of violations and reports within 20 seconds.
- (b) FR-37: The system shall implement secure storage and encryption of student/exam data.
- (c) FR-38: The system shall allow a responsive web interface for investigators and administrators.
- (d) FR-39: The system shall maintain audit logs for all activities.
- (e) FR-40: The system shall support scalability for multiple exam halls simultaneously.

2.3 Non-Functional Requirements

This section specifies the non-functional requirements of the ForeSyte system. These requirements are practical, measurable, and verifiable to ensure the system can be realistically implemented in an academic setting.

2.3.1 Reliability

- REL-1: The system shall achieve at least 95% uptime during scheduled exam sessions.
- REL-2: In case of camera or feed failure, the system shall log the event and notify the administrator within 1 minute.
- REL-3: The system shall store video footage and violation data redundantly to prevent data loss.

2.3.2 Usability

- USE-1: An administrator shall be able to generate a violation report in no more than 3 interactions.
- USE-2: New users (administrators or investigators) shall be able to learn the core functions of the system with less than 2 hours of training.
- USE-3: The dashboard shall present violation summaries in a clear tabular format with filters.

2.3.3 Performance

- PER-1: The system shall process live camera feeds with a maximum latency of 10 seconds.
- PER-2: The cheating detection engine shall analyze at least 20 frames per second per camera feed.
- PER-3: Violation reports shall be generated within 15 minutes of the end of an exam session.

2.3.4 Security

- SEC-1: Only authenticated users shall access the system dashboard.
- SEC-2: User access shall be role-based (administrator, investigator).
- SEC-3: All access attempts shall be logged for audit purposes.

2.3.5 Maintainability

- MAI-1: The system shall follow a modular architecture so that detection models can be updated independently.
- MAI-2: Minor bugs shall be diagnosable and fixable within 48 hours.
- MAI-3: The documentation shall describe system modules, and troubleshooting in detail for developers.

Chapter 3

System Overview [AS PER FYP1 MID REPORT]

ForeSyte AI Exam Surveillance System is an intelligent monitoring solution designed to enhance the integrity of in-person examinations by automating student and invigilator monitoring. The system integrates CCTV cameras, computer vision, and deep learning models to detect suspicious student behaviors such as phone usage, frequent seat movements, or unauthorized communication, while also tracking invigilator activities to ensure accountability. Each activity is mapped to seating plans and verified against student identities using face recognition, preventing impersonation. Detected events are logged as timestamped violations, which are compiled into structured reports accessible through an administrative dashboard. Administrators manage user profiles, seating plans, and exam sessions, while disciplinary investigators utilize the violation reports to review cases and make informed decisions. The system follows a modular architecture consisting of two primary components: the ForeSyte Web Portal, a multi-user web interface for dashboard visualization and report management, and the Backend Service, the core AI engine handling video stream processing, cheating detection, face recognition, and automated report generation. This architectural design ensures separation of concerns, scalability, and maintainability while providing seamless integration between the user interface and the underlying AI processing capabilities.

3.1 Architectural Design

The ForeSyte AI Exam Surveillance System is designed using a modular architecture to ensure scalability, maintainability, and clear separation of concerns. At a high level, the system is divided into front-end modules, back-end modules, and supporting external systems. Each module plays a distinct role but collaborates with others to achieve the

overall goal of automated, AI-powered exam monitoring.

The architecture follows a **two-tier client-server architecture** with **layered components**, implementing a **RESTful API** design pattern. The front-end (presentation tier) handles user input, report visualization, and dashboard interactions through a React-based single-page application. The back-end (application tier) processes video streams, performs AI-based detection and analysis, and manages business logic through FastAPI routes. The data management layer (data tier) ensures persistence and retrieval of information using PostgreSQL for relational data and MongoDB for high-velocity event logs. This decomposition ensures that the AI detection logic, data storage, and user interfaces remain loosely coupled and independently upgradable.

3.3.1 Box and Line Diagram (Initial Design Stage)

In the initial design stage, a simple box and line diagram is used to represent the main system components and their interactions. This diagram provides a high-level overview of how the core modules connect, without detailing the internal architecture or data flow.

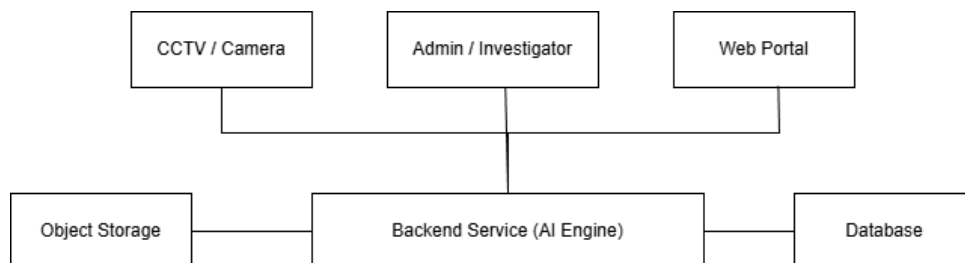
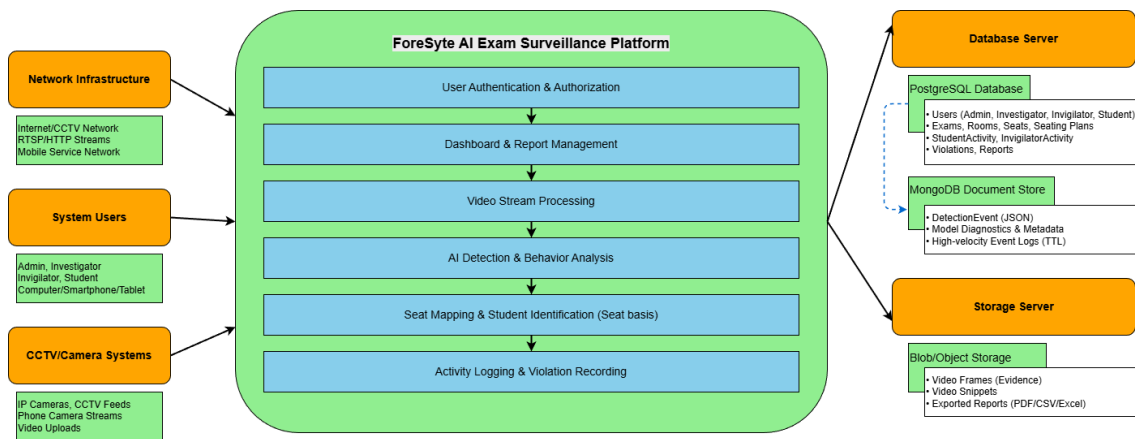


Figure 3.1: Box and Line Diagram for ForeSyte AI Exam Surveillance System

3.3.2 High-Level Architectural Style

After the initial overview, the finalized architecture style/pattern diagram is presented. The system uses a two-tier client-server architecture with layered components, providing clear separation between the presentation layer (React frontend), application layer (FastAPI backend with embedded business logic), and data management layer (PostgreSQL and MongoDB). Figure 3.2 illustrates the high-level architecture of the system, showing the main subsystems and their interactions.

Figure 3.2: High-Level Architectural Design of the ForeSyte AI Exam Surveillance System



3.3.3 Mapping of Modules/Components

Each module/component is mapped to its respective part of the architecture, ensuring modularity and scalability. The architecture supports integration with external systems (CCTV, storage, databases) and provides a robust foundation for AI-powered exam monitoring.

3.4 Data Design

The data design for **ForeSyte** focuses on how the system's information domain is represented and transformed into data structures across relational, document, and object stores. In this section, we outline the major system entities, how they are stored, processed, and managed to ensure robust operation and secure handling of user information and evidence.

3.4.1 Data Structures and Organization

ForeSyte deals with multiple data classes: user identities, seating plans, live/recorded video evidence, AI detections, activities, violations, and reports. The core data is organized so that:

- **Relational (PostgreSQL)** holds normalized, queryable facts: Users, Exams, Rooms, Seats, StudentActivity, InvigilatorActivity, Violations, Reports, and seat assignments; enforced with foreign keys.
- **Document (MongoDB)** holds high-velocity *DetectionEvent* JSON and model diagnostics with TTL, enabling fast ingest and optional replay/debug.
- **Blob/Object Storage** (evidence bucket) stores immutable artifacts: frame crops, video snippets, and exported PDF reports.

Data lifecycle. Streams or uploads produce detection events; an aggregation service converts them into activities and violations with evidence links; dashboards and investigators read relational views and open the referenced evidence in object storage.

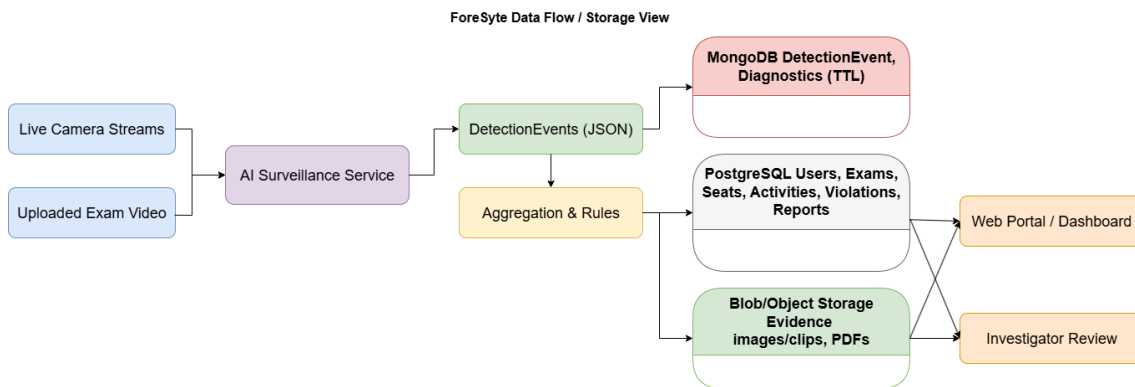
3.4.2 Major Data Entities

Key logical entities:

- **User Information:** Admin, Invigilator, Investigator, and Student profiles with credentials and roles.
- **Exam/Room/Seat:** Exam sessions; rooms with cameras; seats mapped to students and exams.
- **DetectionEvent** (document): Raw detector output with timestamps, camera/room source, and model metadata.
- **StudentActivity/InvigilatorActivity:** Curated activities derived from detections; include severity, confidence, and optional evidence URL.
- **Violation:** Confirmed cheating events; include type, timestamp, severity, status, and optional evidence URL.
- **Report:** Generated summaries (PDF) of violations and activities for investigators.

Similarly, Figure 3.3 shows how streams/uploads are processed and persisted.

Figure 3.3: Data Flow and Storage View (PostgreSQL, MongoDB, Blob/Object Storage)



3.4.5 Data Flow and Processing

The system processes exam footage via two paths that converge into a unified pipeline:

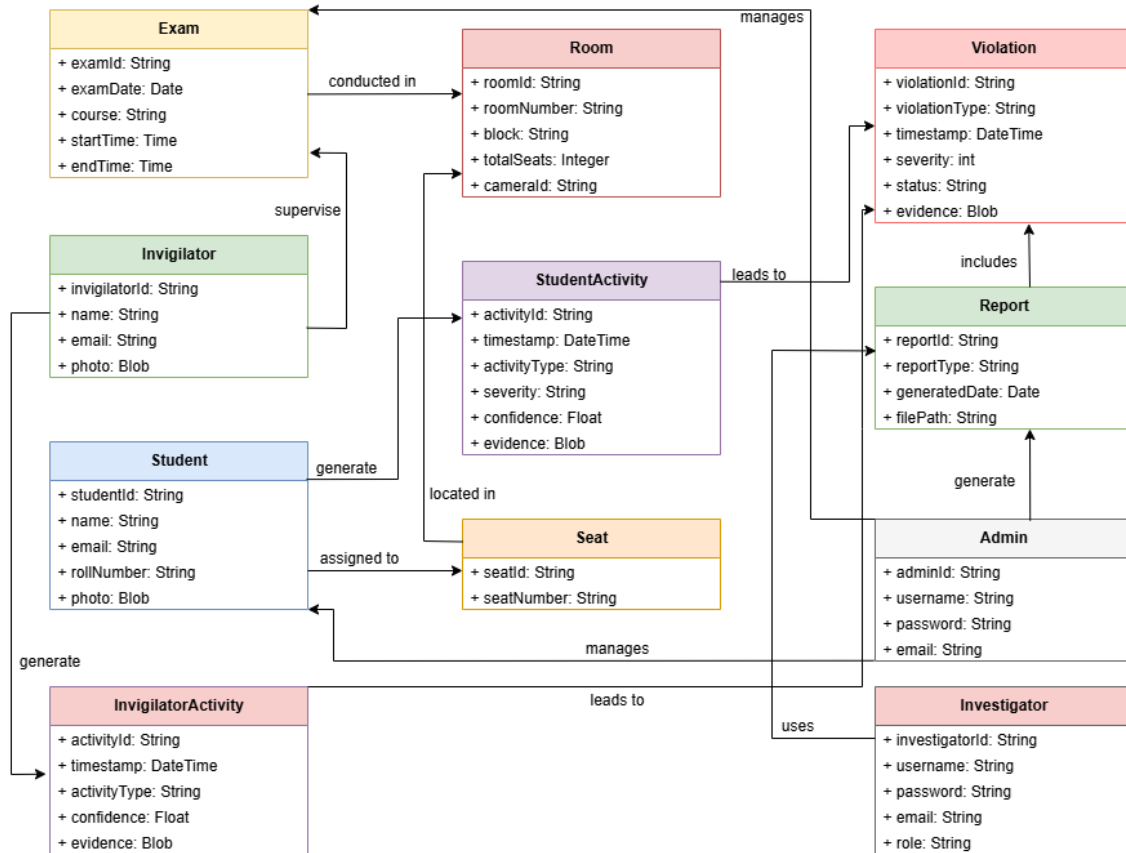
1. **Real-time (Live) Monitoring:** Camera streams feed the AI service, which emits per-frame *DetectionEvents* (JSON).

2. **Uploaded Footage:** After the exam, recorded video is ingested; the AI service produces the same *DetectionEvents* offline.
3. **Aggregation/Rules:** Business rules transform events into *StudentActivity/InvigilatorActivity*; suspicious patterns yield *Violation* rows with evidence links.
4. **Persistence:** Normalized facts land in PostgreSQL; evidence media is stored in Blob/Object storage; optionally, transient raw events remain in MongoDB (TTL).
5. **Consumption:** Admin dashboards query relational views; investigators open pre-signed evidence URLs and download reports.

3.2 Domain Model

Figure 3.4 represents the domain model for ForeSyte.

Figure 3.4: ForeSyte Domain Model Diagram



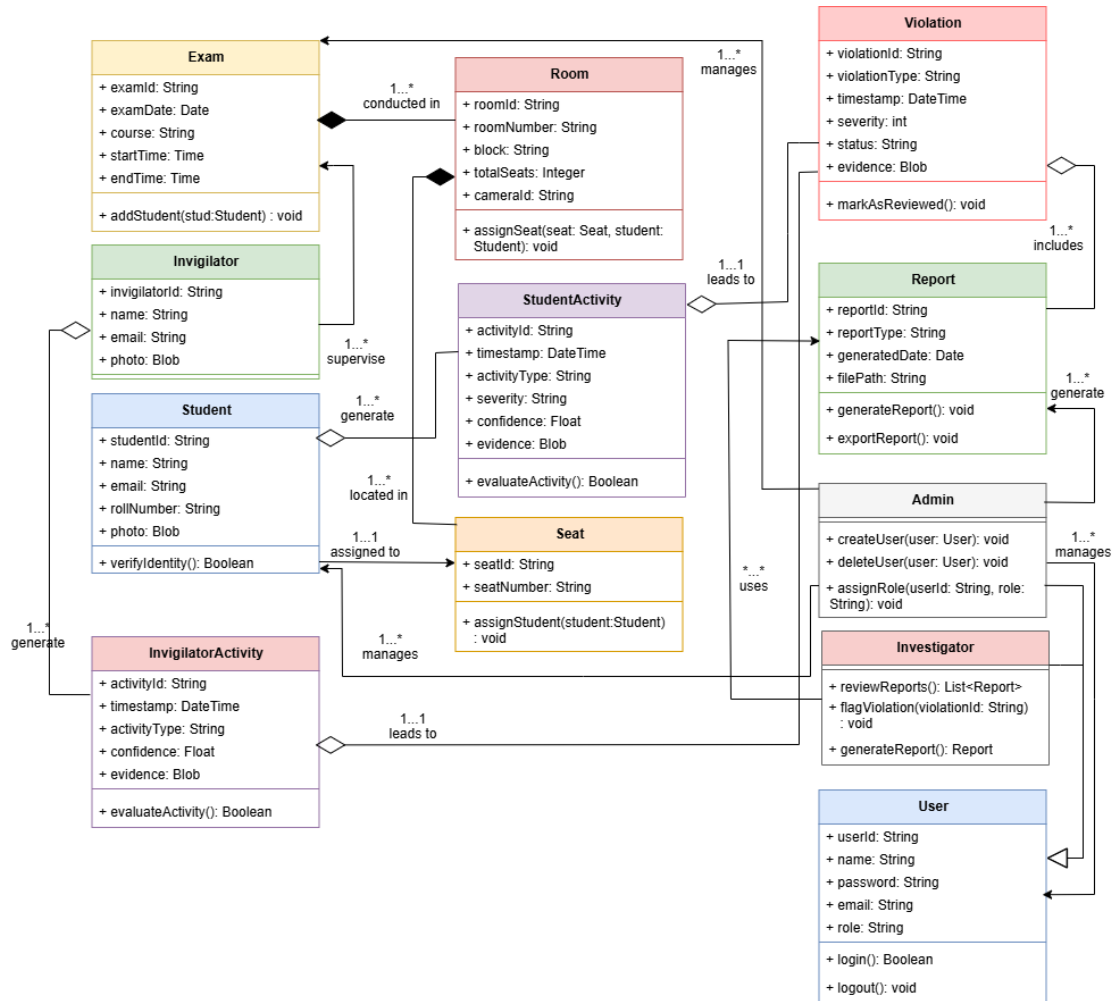
3.3 Design Models [UPTO THE CURRENT ITERATION ONLY]

The ForeSyste system employs an object-oriented development approach, utilizing several key design models to represent the system's structure, behavior, and interactions. These models provide comprehensive documentation of the system's architecture and facilitate understanding of complex relationships between system components.

3.3.1 Class Diagram

The system's class diagram represents the core entities and their relationships within the ForeSyste domain model.

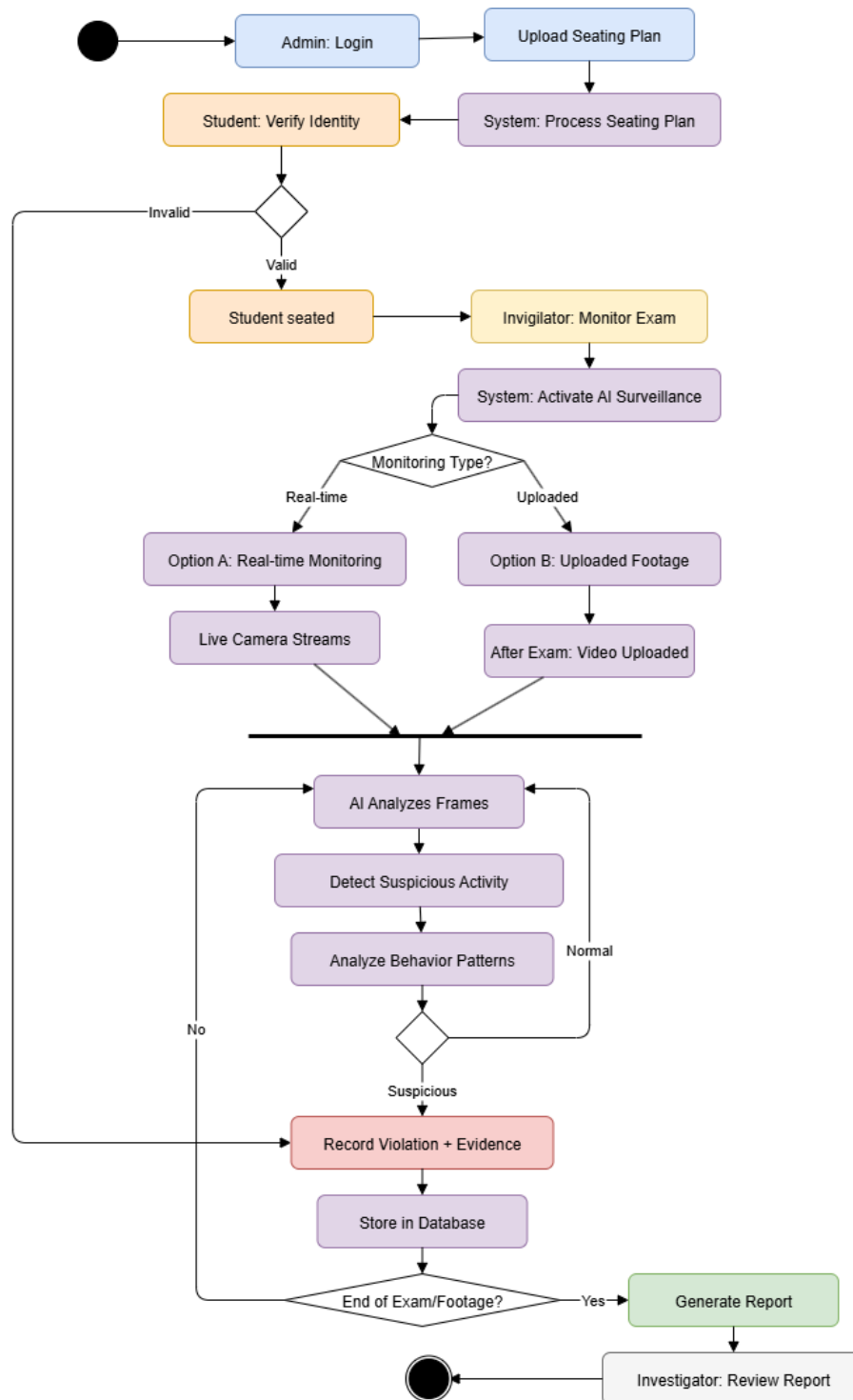
Figure 3.5: ForeSyste Class Diagram



3.3.2 Activity Diagram

The system's activity diagram illustrates the main processes and workflows:

Figure 3.6: ForeSyte Activity Diagram



3.2.3 Sequence Diagrams

This section presents the sequence diagrams for all identified use cases (UC-01 to UC-07). Each diagram illustrates the dynamic interaction between actors, the ForeSys AI System, and domain entities.

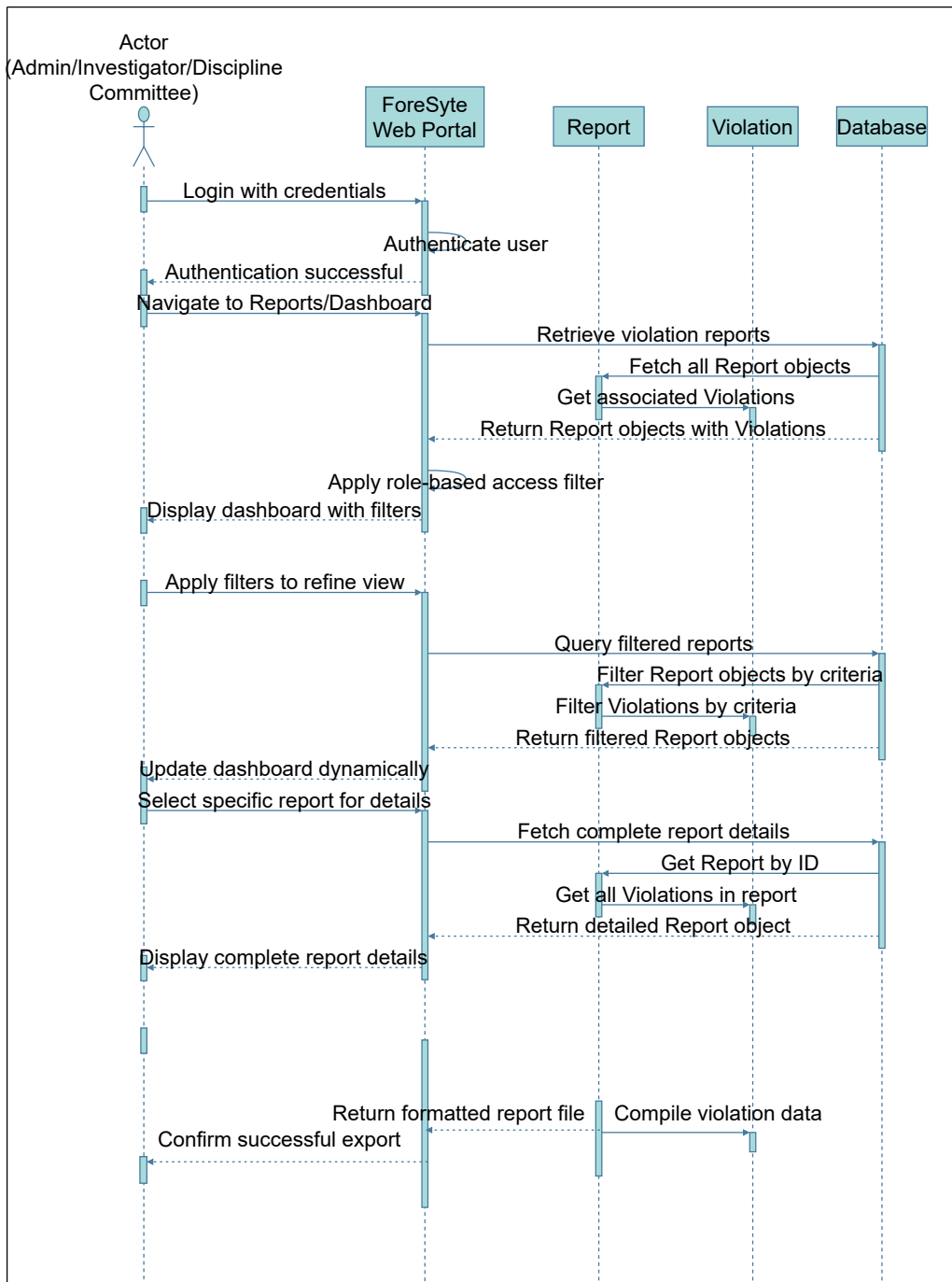


Figure 3.7: Sequence Diagram for UC-01: View and Manage Reports/Dashboard

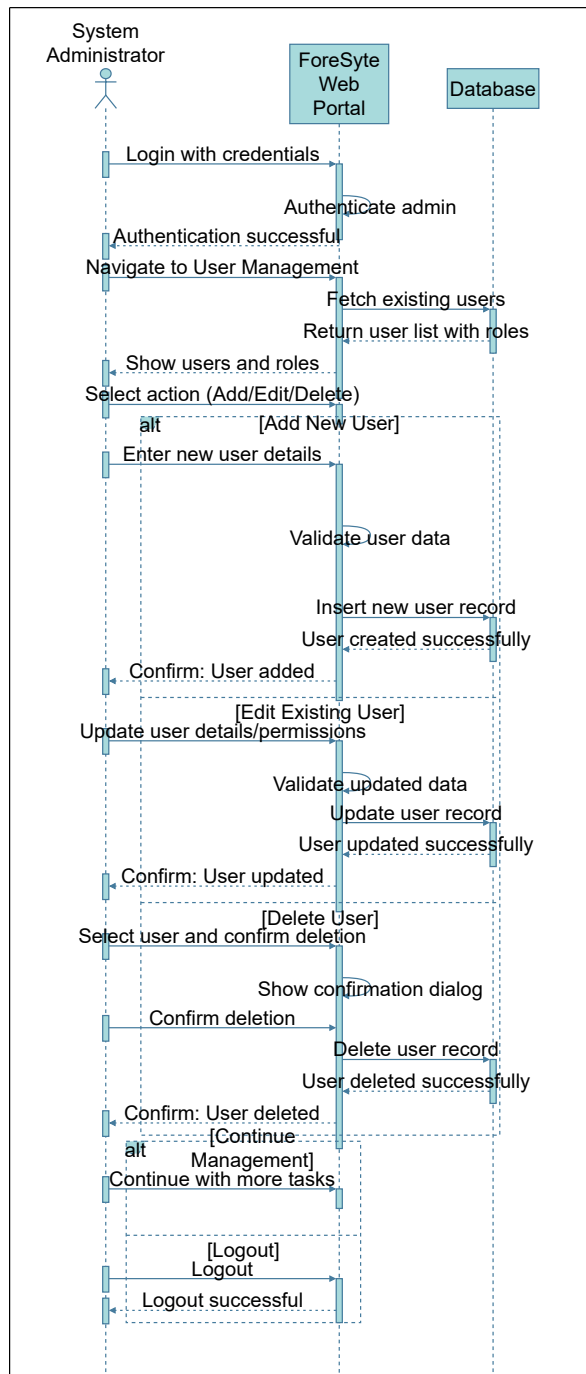


Figure 3.8: Sequence Diagram for UC-02: Manage User Profiles

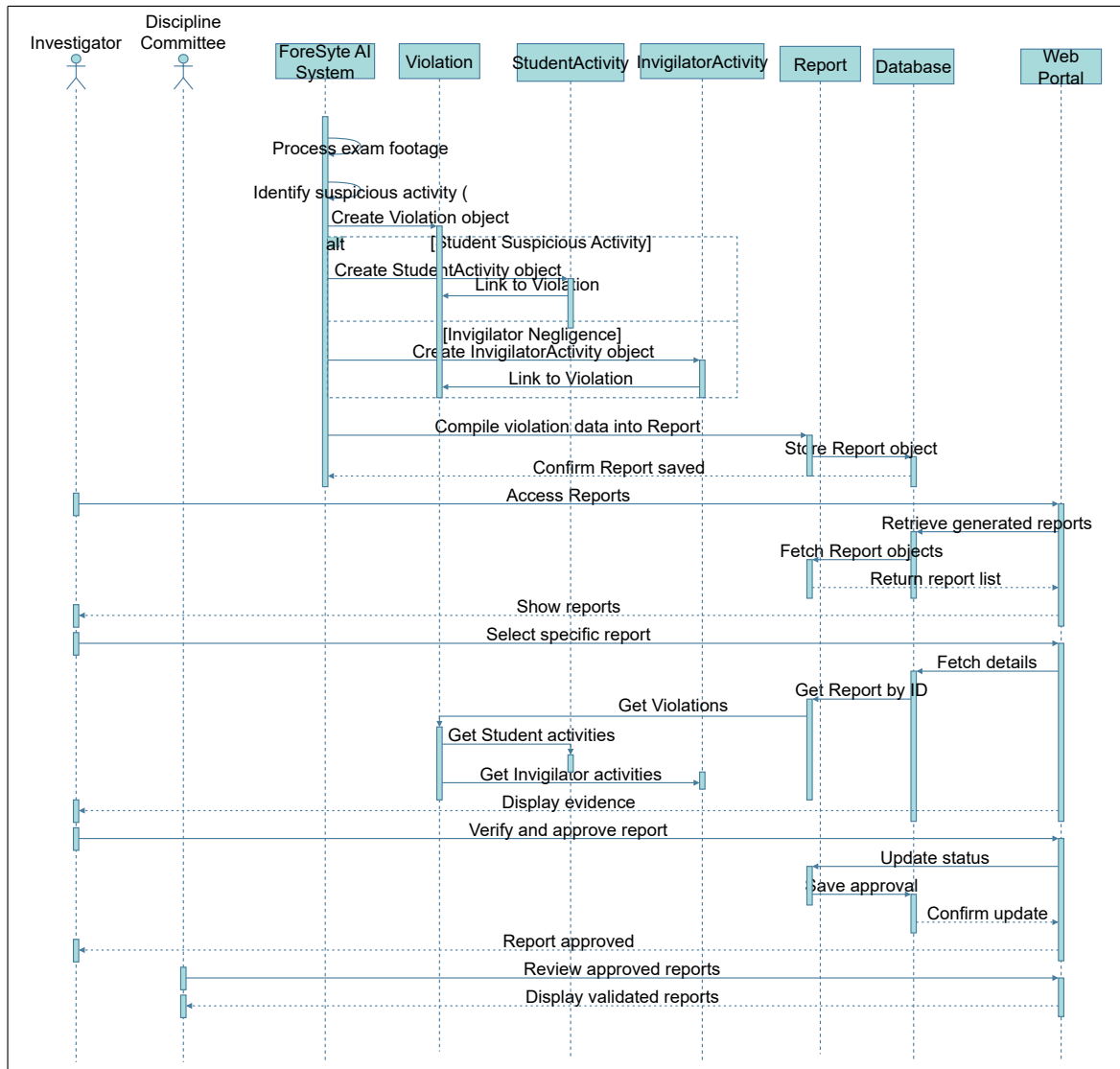


Figure 3.9: Sequence Diagram for UC-03: Generate Detection Reports

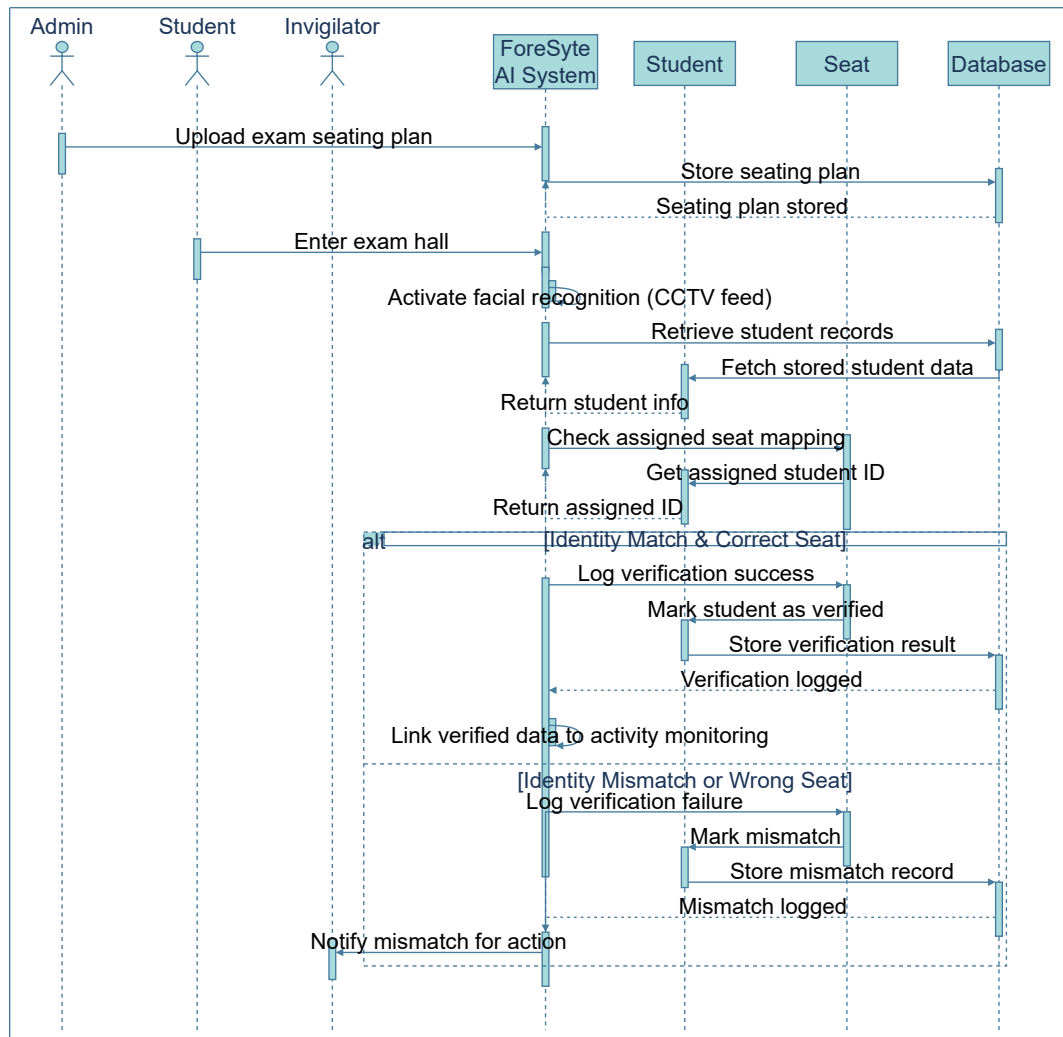


Figure 3.10: Sequence Diagram for UC-04: Verify Student Identity & Seat Mapping

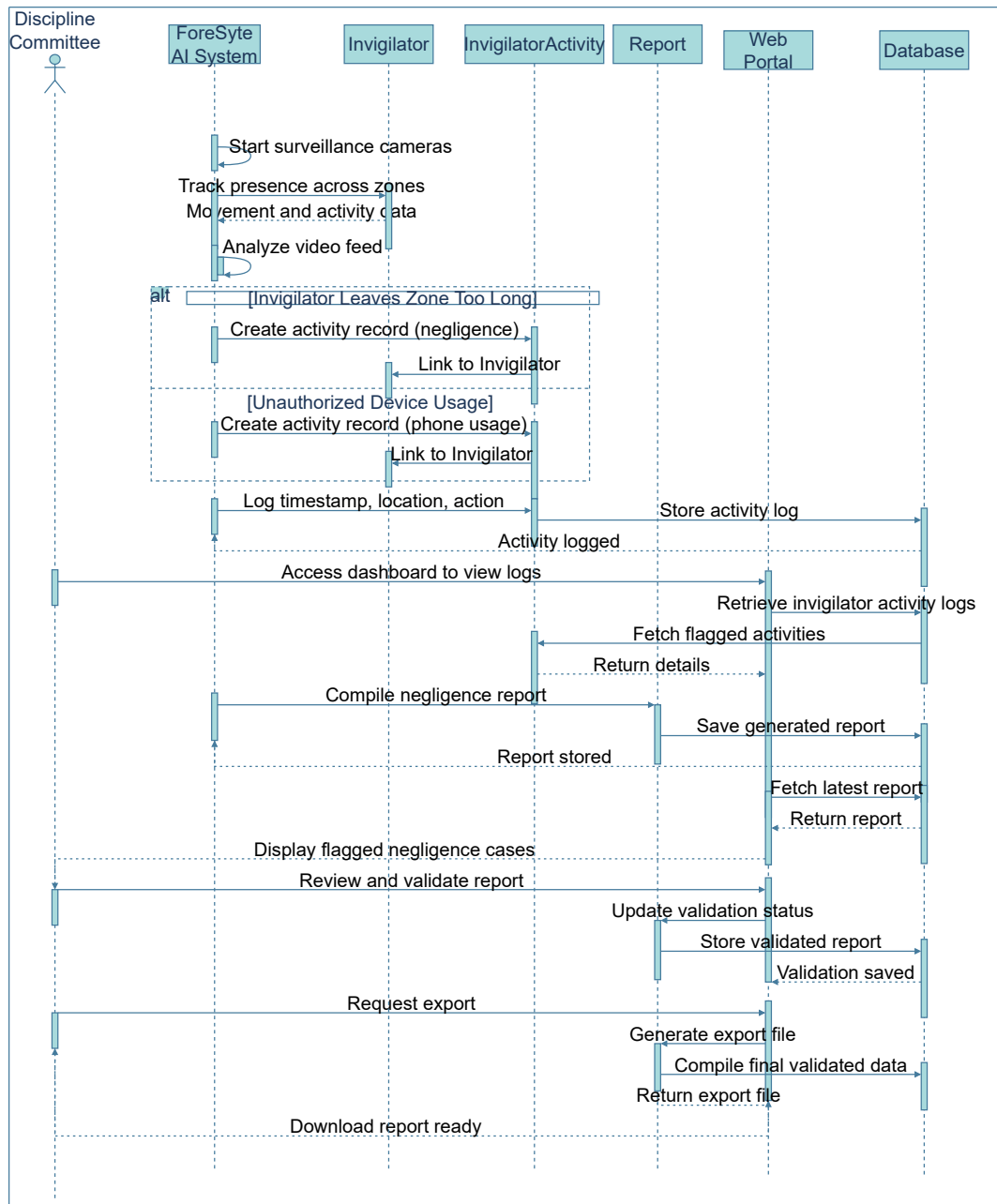


Figure 3.11: Sequence Diagram for UC-05: Monitor Invigilator Activity

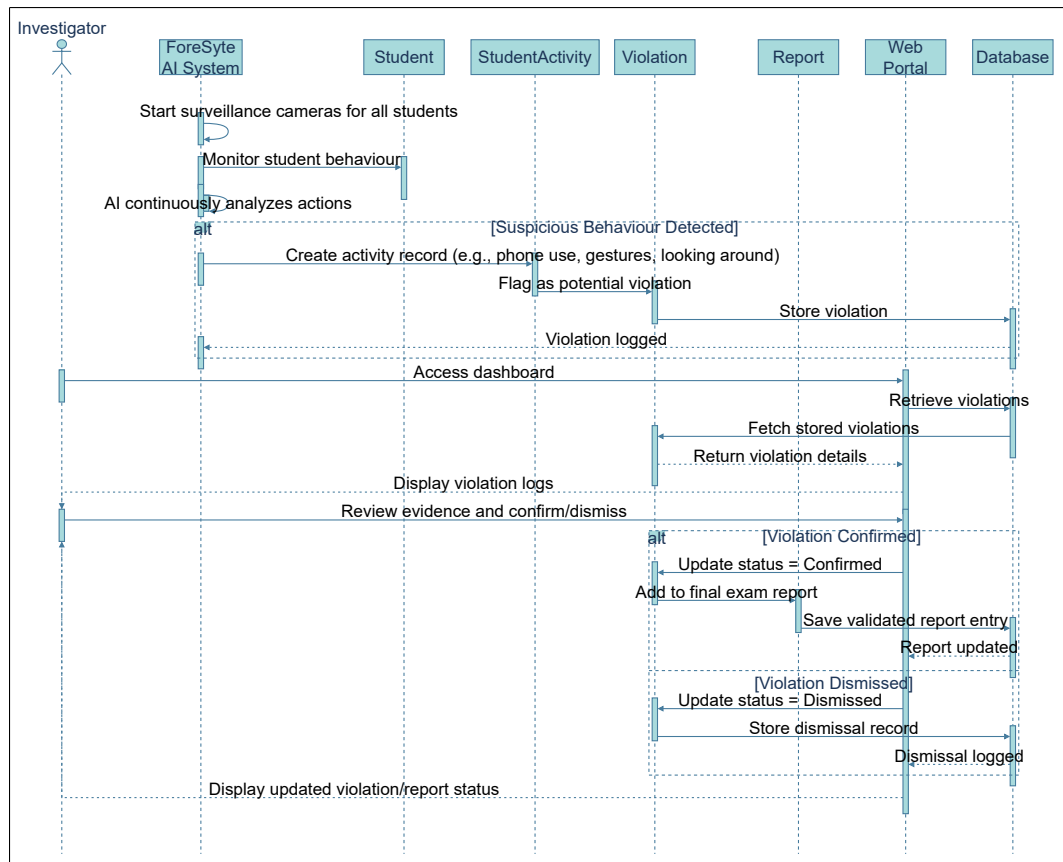


Figure 3.12: Sequence Diagram for UC-06: Monitor Student Behaviour

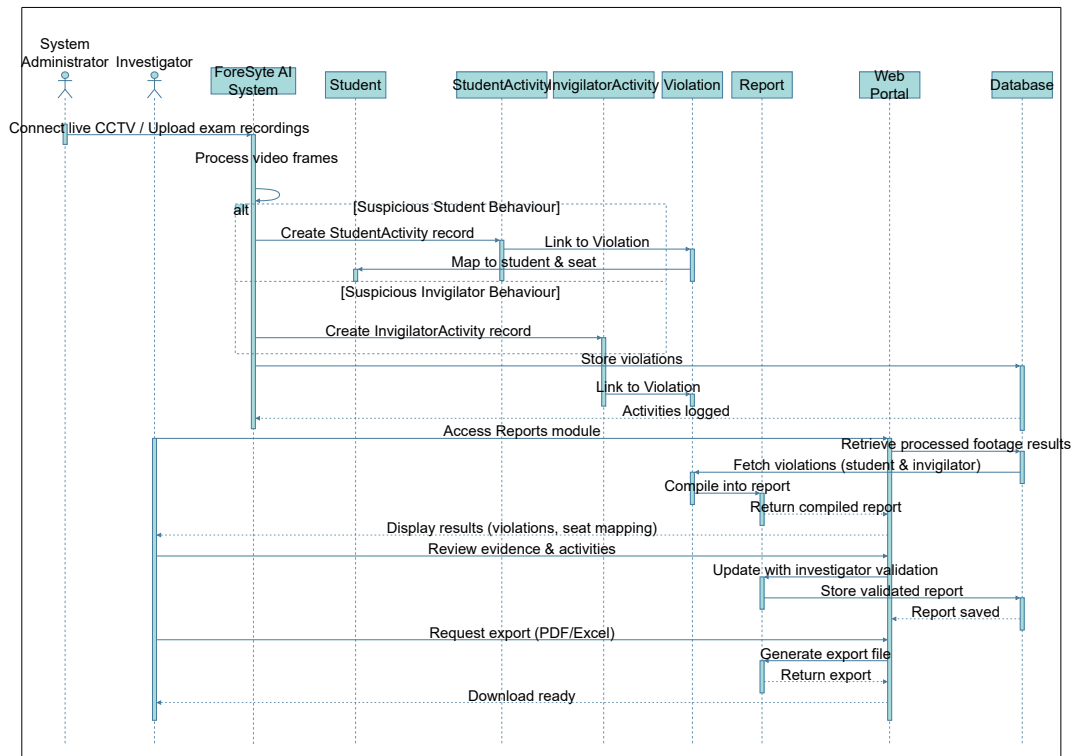


Figure 3.13: Sequence Diagram for UC-07: Process Exam Footage (Live/Recorded)

3.3.3 State Transition Diagram

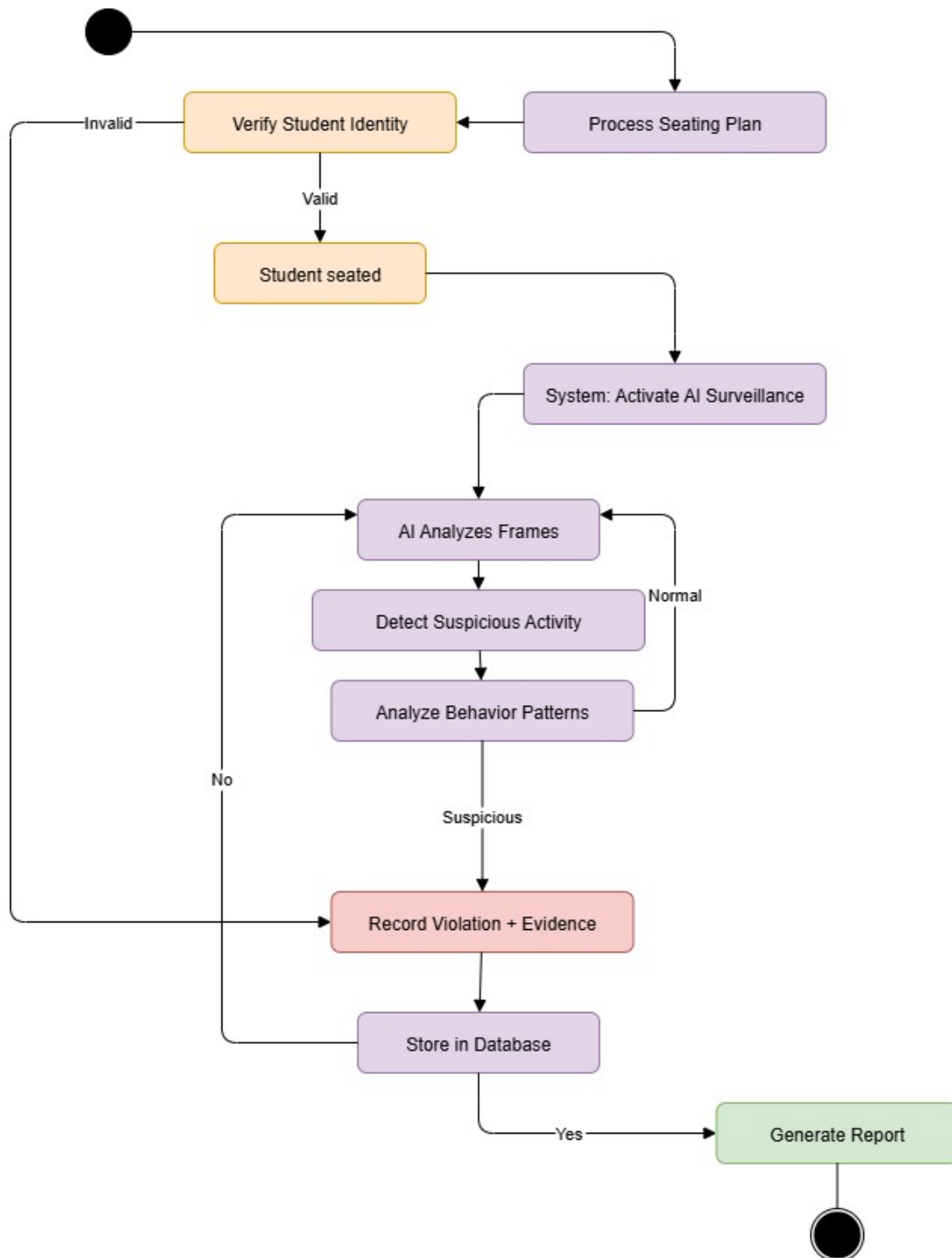


Figure 3.14: State Transition Diagram

This model collectively provide a comprehensive view of the ForeSyte system's architecture, ensuring clear understanding of system components, their interactions, and the overall

system behavior throughout the examination monitoring process.

Chapter 4

Implementation and Testing

This chapter describes the current implementation status of the **ForeSyte** AI Exam Surveillance System and the testing performed up to this iteration. The focus of this phase is on the **Admin** and **Investigator** dashboards and their supporting backend services. Computer-vision based monitoring and the Student Portal are planned components and therefore remain out of scope for this iteration.

4.1 Current Progress

4.1.1 Front-End

The front-end of ForeSyte is implemented as a modern single-page application using React, TypeScript, Vite, and Tailwind CSS. The interface is split into role-specific areas so that administrators and investigators see different dashboards and workflows.

Admin Dashboard

The main admin dashboard presents a high-level overview of system activity. It displays:

- **Active Exams** – number of exams scheduled for the current or future dates.
- **Incidents Detected** – count of student activity records (currently using placeholder data; actual cheating detection is planned for future iterations).
- **Students Monitored** – number of students currently under surveillance based on seating plans and exam sessions.
- **Average Exam Duration** – average exam length computed from start and end times.

These statistics are fetched via the backend `/dashboard/stats` endpoint with a period parameter (today, week, or month). The dashboard also includes:

- An **Activity Overview** bar chart that compares the number of incidents and exams across time buckets using the `/dashboard/activity` endpoint.
- A **Recent Incidents** panel that lists the latest flagged student activities with severity tags using the `/dashboard/recent-incidents` endpoint.
- A **Quick Actions** area that links to live monitoring, reports, user management, and system settings.
- A **System Status** section that summarizes the health of the detection pipeline, camera network, analytics, and database.

The Admin Dashboard is therefore fully wired to the backend aggregation APIs. Loading states, error handling, and period switching are implemented in the front-end.

Investigator Portal

The Investigator Portal is implemented as a separate area in the React application, with its own header and sidebar components to clearly separate workflows from the admin role.

- The **Investigator Dashboard** displays summary cards for total violations, pending reviews, violations resolved today, and active exams. It also lists recent activities (e.g., phone usage, invigilator negligence, unauthorized movement) with severity and status badges and a period filter (today, week, month).
- The **Reports Page** shows a grid of generated reports with metadata such as report type (incident/exam), exam name, number of violations, generation date, and author. It provides filters for text search and report type, and a modal workflow for configuring a new report (type, export format, inclusion of video evidence links).

At this iteration, these investigator views use mock data to validate layout and interactions while backend report listing endpoints are being finalized. However, the UI contracts, filter behaviour, and state management are implemented and ready to be connected to live APIs.

4.1.2 Back-End

The backend is built with FastAPI and SQLAlchemy on top of PostgreSQL, with MongoDB used for flexible logging and high-volume events. Authentication uses JWT-based tokens

with role-based access control (RBAC) to differentiate between Admin, Investigator, Invigilator, and Student users.

Two backend modules are particularly important for this iteration:

- **Dashboard API** (/dashboard): aggregates data for the Admin and Investigator dashboards.
- **Reports API** (/reports): manages report generation and retrieval for incidents and exams.

Dashboard API

The /dashboard/stats endpoint computes:

- **Active Exams:** exams scheduled for the current or future dates.
- **Incidents Detected:** count of StudentActivity records in the selected period (note: actual cheating detection via computer vision is not yet implemented; this metric reflects activity records that would be populated by the detection pipeline in future iterations).
- **Students Monitored:** distinct count of students with seat assignments in active exams.
- **Average Exam Duration:** average duration (in minutes) based on start and end times of exams in the selected period.
- **Trends:** percentage changes for incidents, students, and exams when compared to the previous period.

The /dashboard/activity endpoint returns a sequence of time buckets, each containing the number of incidents and exams in that interval. For today, 4-hour slots are used; for week and month, daily buckets are used. The /dashboard/recent-incidents endpoint returns the most recent student activities, joined with student information to show names, types, severity levels, timestamps, and model confidence values.

All dashboard endpoints enforce RBAC policies and only allow access to users with admin or investigator roles.

Reports API

The /reports router provides the API structure for managing report records. **Note:** The actual file generation (PDF, CSV, Excel) is not yet implemented in this iteration; the endpoints currently create database records and return metadata, but do not produce downloadable report files.

- Standard CRUD endpoints (POST /, GET /, GET /{id}, PUT /{id}, DELETE /{id}) operate on the Report model and validate related Violation and Investigator records. These endpoints are functional for managing report metadata.
- POST /reports/incidents accepts a list of incident IDs, desired export format, and whether to include video links. It validates and resolves the incident IDs into StudentActivity records, ensures there is an associated Violation, creates a new Report entry in the database, and returns a JSON response containing the report ID, file path, format, and status. **The actual report file generation (PDF/CSV/Excel) is planned for a future iteration.**
- POST /reports/exams/{exam_id} is designed to generate an exam-level report by aggregating all StudentActivity entries for the given exam and producing a Report record. **Currently, this endpoint creates the database record but does not generate the actual report file.**

These APIs establish the data model and workflow structure for report management, but the actual document generation logic will be implemented in subsequent iterations.

4.1.3 Next Steps

The completed work in this iteration establishes the core dashboard infrastructure and API structure for reports. Upcoming work will:

- Connect the Investigator Portal to live dashboard and report APIs instead of mock data.
- Implement actual report file generation (PDF, CSV, Excel) for incident and exam reports.
- Integrate the video processing pipeline (camera ingestion, AI models, and real-time cheating detection) into the monitoring endpoints.
- Finalize and expose the Student Portal, including student-facing views of violation histories and exam schedules.

4.2 Algorithm Design

This section summarizes the main algorithms and data flows used to support the implemented features in ForeSyte.

4.2.1 Admin Dashboard Aggregation

The Admin Dashboard relies on server-side aggregation logic to compute statistics for different time windows. The high-level algorithm for computing dashboard statistics is:

1. **Determine Time Window:** Given `period ∈ {today, week, month}`, compute the current window (`start_date`) and the immediately preceding window (`prev_start`, `prev_end`) for trend comparison.
2. **Count Active Exams:** Query Exam where `exam_date` is greater than or equal to the current date. Perform the same query on the previous window to obtain `prev_active_exams`.
3. **Count Incidents and Students:**
 - Incidents: count `StudentActivity` records with timestamps within the current window.
 - Previous incidents: count `StudentActivity` records within the previous window.
 - Students monitored: count distinct `Seat.student_id` in rooms and exams that fall inside the current and previous windows.
4. **Compute Average Exam Duration:** For all exams in the current window with non-null start and end times, compute the duration in minutes and average the result.
5. **Compute Trends:** For each metric (incidents, students, exams), compute percentage change between current and previous counts, handling the edge case when the previous value is zero.
6. **Return Structured Response:** Package all calculated values into a `DashboardStats` object and return it to the client.

On the client side, the React dashboard:

- Calls `/dashboard/stats` to populate the metric cards.
- Calls `/dashboard/activity` to render a bar chart comparing incidents and exams per time bucket.
- Calls `/dashboard/recent-incidents` to display the latest suspicious activities with relative time labels.

4.2.2 Report Generation Workflow (Planned Design)

The report generation workflow is designed to be configurable and format-agnostic. **Note:** The following describes the planned design; actual file generation (PDF, CSV, Excel) is not yet implemented in this iteration.

Incident Reports (Planned)

1. The investigator selects one or more incident IDs and chooses an export format (e.g., PDF, CSV, JSON), with options such as including video evidence links.
2. The front-end constructs an `IncidentReportRequest` and sends it to `POST /reports/incidents`.
3. The backend validates permissions, resolves the incident IDs into `StudentActivity` records, ensures there is an associated `Violation`, and generates a unique report filename.
4. A new `Report` record is inserted in the database, and the API returns the report ID, file path, format, and status to the client. **In the current iteration, the actual report file is not generated; only the database record is created.**

Exam Reports (Planned)

1. The investigator selects an exam and desired output format from the Reports Page.
2. The front-end sends an `ExamReportRequest` to `POST /reports/exams/{exam_id}`.
3. The backend verifies the exam, aggregates its related `StudentActivity` records, ensures a linked `Violation` exists, and creates a `Report` entry in the database.
4. The API responds with the report metadata. **The actual report file generation (PDF/CSV/Excel) will be implemented in a future iteration.**

4.2.3 Investigator Violation Review Flow

The Investigator Dashboard is designed to give investigators a structured way to triage violations:

1. Load dashboard summary counters (total violations, pending reviews, resolved today, active exams).
2. Display a list of recent activities with type, severity, and status tags.
3. Allow investigators to toggle the period filter to focus on today, week, or month.

4. Provide a *Review* action for each activity which, in the next iteration, will navigate to a detailed case view and allow updating violation status and adding investigation notes.

4.2.4 Seating Plan Upload and Processing

The seating plan upload algorithm processes PDF files containing exam schedules and extracts room, student, and seat assignment information. The algorithm handles PDF parsing, data extraction, database operations, and visualization generation. The high-level pseudocode for this process is:

```
START timer
```

```
READ uploaded PDF bytes
```

```
EXTRACT full text from all pages
```

```
NORMALIZE text (remove extra whitespace)
```

```
DEFINE block_pattern to detect each exam block inside PDF
```

```
FIND all exam blocks using regex
```

```
CREATE empty list: matching_exams
```

```
FOR each exam block:
```

```
    EXTRACT exam date
```

```
    EXTRACT exam time slot
```

```
    EXTRACT room number
```

```
    IF failed → continue
```

```
    CHECK if user-specified room_no matches (if provided)
```

```
    CHECK if user-specified time_slot matches (if provided)
```

```
    IF mismatch → continue
```

```
    EXTRACT course name + section
```

```
    EXTRACT invigilator name
```

```
FIND all students using regex:
```

```
    serial_no, roll_no, name, seat_no
```

```
ADD exam entry to matching_exams list:
```

```
    date, time, course, section, room, invigilator,
```

```
        student_list, total_students, uploaded_at

IF matching_exams is empty:
    RETURN "No seating plan found"

SELECT first exam → selected_exam
STORE selected_exam in global latest_room_data

PARSE exam_date, start_time, end_time using parse_date_time()

-----
DATABASE OPERATIONS
-----

# --- EXAM ---
QUERY Exam table:
    WHERE course == selected_exam.course
    AND    exam_date == exam_date
    AND    start_time == start_time

IF exam exists:
    use existing
ELSE:
    CREATE new Exam object
    COMMIT to DB

# --- ROOM ---
Split room number into block + room_num
QUERY Room table:
    WHERE room_number == room_num AND exam_id == exam.exam_id

IF room exists:
    use existing
ELSE:
    CREATE Room with:
        room_number, block, total_seats, exam_id, camera_id
    COMMIT to DB

# --- STUDENTS + SEATS ---
FOR each student in selected_exam.students:
```

```

roll_number = student.roll_no
name = student.name

QUERY Student table WHERE roll_number matches
IF not exists:
    CREATE new Student(name, email_from_roll, roll_number)
    COMMIT

seat_number = student.seat_no

QUERY Seat table WHERE (room_id, seat_number)
IF exists:
    UPDATE seat.student_id
ELSE:
    CREATE new Seat(room_id, seat_number, student_id)

COMMIT all changes

-----
SAVE JSON EXPORT
-----

GENERATE timestamp-based filename
SAVE selected_exam as JSON file in extractions folder

-----
VISUALIZATION PROCESS
-----

LOAD CCTV frame image
LOAD seat_map.json (polygon coordinates)

CALCULATE max_column in seating plan:
    For each student seat (C#R#)
        Extract column number
        Track the maximum found

DEFINE mapping rules based on max_column:
    if max_col == 6 → map 6 columns to 1,3,5,6,8,10
    if max_col == 5 → map 5 columns to 1,4,6,8,10

```

```
    else → identity mapping

DEFINE function map_seat_to_seat_map(seat_no):
    Extract C#, R#
    Convert input col → mapped column
    RETURN seat_map ID like seat_cXrY

CREATE:
    filled_seats = all seat IDs containing students
    student_seat_map = map seat_id → student

FOR each seat_id in seat_map:
    IF seat_id NOT in filled_seats → skip

    GET polygon points
    DRAW polygon in green

    FIND student assigned
    CALCULATE centroid
    DRAW student name + roll number at centroid

SAVE annotated image in extractions folder

-----
RETURN RESPONSE
-----

RETURN:
    message
    exam_date
    exam_time
    course
    room_no
    students_count
    exam_id
    room_id
    json_file path
    annotated_image path
    processing_time
```

The algorithm processes seating plan PDFs through several stages: (1) **PDF Parsing and Text Extraction**, where the uploaded PDF is read and normalized; (2) **Pattern Matching**, where exam blocks are identified using regex patterns and filtered by user-specified room number and time slot; (3) **Data Extraction**, where course information, invigilator details, and student seat assignments are extracted; (4) **Database Operations**, where Exam, Room, Student, and Seat records are created or updated; (5) **JSON Export**, where the extracted data is saved as a JSON file; and (6) **Visualization**, where the seating plan is overlaid on a CCTV frame image with student names and roll numbers annotated at their respective seat positions.

4.3 External APIs and Libraries

At this stage, ForeSyte does not depend heavily on third-party AI services, but it makes extensive use of framework libraries and SDKs:

- **FastAPI and Starlette** for HTTP routing, dependency injection, and request handling.
- **SQLAlchemy** and **psycopg2** for ORM and PostgreSQL connectivity.
- **PyMongo** for MongoDB integration in the logging and evidence pipeline.
- **OpenCV** and image processing libraries, which are already installed and used in internal test scripts to handle frames for future AI detectors.
- **React Router**, **Tailwind CSS**, and **lucide-react** on the front-end for routing, styling, and icons.

4.4 Testing Details

Testing for this iteration concentrates on validating backend APIs, role-based access control, and the correctness of aggregated statistics, along with front-end behaviour under typical and edge-case conditions.

4.4.1 Unit Testing

Reports API Tests

Table 4.1 summarizes representative unit tests for the Reports API.

Table 4.1: Unit Test Cases for Reports API

Test ID	Object	Preconditions	Steps	Test Data	Expected Result	Postconditions	Actual Result	Pass/Fail
TC201	Verify admin can create a report	Valid admin JWT; existing Violation and Investigator records	Call POST /reports with valid payload	report_type: "incident"				
violation_id: valid UUID generated_by: valid UUID	Return 201 Created and a Report row object with non-null report_id	Database contains new Report row	As Expected	Pass				
TC202	Enforce RBAC on report creation	Authenticated investigator JWT	Call POST /reports with valid payload	Same as TC201	Returns 403 Forbidden with "Only admins can create reports"	No new Report row is created	As Expected	Pass
TC203	Create report record for incidents (file generation not yet implemented)	Investigator JWT; at least one Student ID present	Call POST /reports with valid payload	incident_ids: [valid UUIDs]				
format: "pdf" include_video_links: true	Return JSON with non-empty id	New Report row linked to incident	As Expected	Pass				

Dashboard API Tests

Table [4.2](#) lists key unit tests for the dashboard statistics and recent incidents endpoints.

Table 4.2: Unit Test Cases for Dashboard APIs

Test ID	Objective	Preconditions	Steps	Test Data	Expected Result	Postconditions	Actual Result	Pass/Fail
TC301	Retrieve “today” dashboard stats	Admin JWT; at least one exam scheduled today	Call GET /dashboard/stats	period: "today"	Returns a valid object; all counts are non-negative and consistent with seeded data	Values computed based on sample database state	As Expected	Pass
TC302	Verify period filter behaviour	Seeded data across multiple days	Call GET /dashboard/stats with different periods	period: "today", "week", "month"	Returns different counts reflecting the date range; no errors occur	Internal trend computation behaves correctly	As Expected	Pass
TC303	Enforce RBAC on stats	Student JWT	Call GET /dashboard/stats	period: "today"	Returns 403 Forbidden and “Access denied”	No sensitive data are exposed	As Expected	Pass
TC304	Return recent incidents list	Student JWT; rows present in DB	Call GET /dashboard/recent-incidents	limit: 5	Returns up to 5 incidents ordered by timestamp; each item includes ID, student name, type, severity, time at	Activities are ordered from most recent to oldest	As Expected	Pass

Front-End Behaviour Tests

The front-end was validated using component-level tests and manual exploratory runs:

- **Admin Dashboard Rendering:** Ensured that the metric cards, activity chart, and recent incidents list render correctly when provided with sample API responses, and that loading and empty states are displayed when data is unavailable.
- **Investigator Dashboard Layout:** Checked that period filters toggle correctly, summary counters are displayed, and recent activity items show appropriate severity and status chips.
- **Reports Page Filtering:** Verified that text search and report-type filters reduce the visible set of reports appropriately and that the “Generate New Report” modal captures the selected configuration.

In addition, a dedicated test script was developed for testing the phone stream ingestion and processing pipeline without enabling AI-based detection. This script validates connection to phone camera streams and basic frame capture, ensuring that the underlying streaming components are stable and ready for integration with the full monitoring pipeline in subsequent iterations. **Note:** Actual cheating detection via computer vision models is not yet implemented; the test script focuses on connectivity and frame handling only.

User Management API Tests

Table 4.3 summarizes unit tests for the User Management API, which allows administrators to create, view, update, and delete users across all roles (Admin, Investigator, Invigilator, Student).

Table 4.3: Unit Test Cases for User Management API

Test ID	Object	Precondition	Setup	Test Data	Expected Result	Postcondition	Actual Result	Pass/Fail
TC401	Verify admin can create a new student user	Valid admin JWT token with access to student data	Call POST /users	name: "John Doe"				
email: "john@example.com" user_type: "student" roll_number: "22I-1234" password: "secure123!"								
	Return 201 Created with User in object database containing non-null id, password, email, and roll_number	New Student record exists in database with non-null id, password, email, and roll_number	As Expected	Pass				
TC402	Verify admin can create an investigator user	Valid admin JWT token with access to investigator data	Call POST /users	name: "Jane Smith"				
email: "jane@example.com" user_type: "investigator" designation: "Senior Investigator" password: "secure123!"								
	Return 201 Created with User in object database containing designation field	New Investigator record exists in database containing designation	As Expected	Pass				
TC403	Enforce RBAC	Authenticated Client	Call POST /users	Same	Return 403	No	As Expected	Pass

Seating Plan Upload Tests

Table 4.4 lists test cases for the seating plan upload functionality, which processes PDF files containing exam schedules and extracts room, student, and seat assignment information.

Table 4.4: Unit Test Cases for Seating Plan Upload

Test ID	Object	Pre-conditions	Steps	Test Data	Expected Result	Test Condition	Actual Result	Pass/Fail
TC501	Upload valid PDF seating plan with PDF JWT; PDF with database room accessible filter	Valid Call	POST file: seating_plan.pdf; /upload-seating-plan	Return 200 OK with extracted data including room number, exam date, time slot, and list of students with seat numbers	Room As Expected Student, and Seat data records created/updated in database	Pass		
TC502	Upload seating plan with time slot filter	Valid Call	POST file: seating_plan.pdf; /upload-seating-plan	Return 200 OK with data for	Only As Expected exam data is extracted	Pass		

Monitoring and Camera Integration Tests

Table 4.5 presents test cases for the monitoring endpoints and camera integration functionality, which enable administrators and investigators to view live camera feeds and manage monitoring sessions.

Table 4.5: Unit Test Cases for Monitoring and Camera Integration

Test ID	Object	Preconditions	Steps	Test Data	Expected Result	Postconditions	Actual Result	Pass/Fail
TC601	Retrieve monitoring feeds for all cameras	Admin JWT; rooms with cameras exist in database	Call GET /monitoring/	No parameters	Returns 200 OK. Feeds valid with list of camera ID, room name, status, stream URL, and students monitored count	Response contains valid feed data for all rooms including camera	As Expected	Pass
TC602	Filter monitoring feeds by exam ID	Admin JWT; multiple exams with rooms	Call GET /monitoring/	exam_id={valid UUID}	Returns 200 OK. Only feeds for rooms associated with the specified exam	Response contains feeds filtered by exam_id={exam_id}	As Expected	Pass
TC603	Enforce RBAC on monitoring feeds	Student JWT	Call GET /monitoring/	Student token	Returns 403 Forbidden with “Access denied”	No monitoring feeds exposed to unauthorized users	As Expected	Pass
TC604	Verify camera status endpoint	Admin JWT; camera exists	Call GET /monitoring/camera/	camera valid ID	Returns 200 OK. Camera valid with status	Response contains valid {camera_id}/status	As Expected	Pass

Bibliography

- [1] Saravanan Arumugam. Deep learning-based smart invigilation system for enhanced exam integrity. *Proceedings of Engineering and Technology Innovation*, 29:99–115, 2025.
- [2] A. Bhatti and T. Aslam. Smart exam monitoring system using computer vision. *Journal of Emerging Technologies*, 2023.
- [3] A. Bucciol, S. Cicognani, and N. Montinari. Cheating in university exams: the relevance of social factors. *ResearchGate*, 2020.
- [4] M. Khalid and H. Shah. Impact of surveillance systems on academic integrity in higher education. *Journal of Educational Research*, 2021.
- [5] Mohammad M. Masud, Khadim Hayawi, Sujith Samuel Mathew, Temesgen Mikael Abraha, and Mai ElBarachi. Smart online exam proctoring assist for cheating detection. *ResearchGate*, 2025.
- [6] Reuben Moyo, Stanley Ndebvu, Michael Zimba, and Jimmy Mbelwa. A video-based detector for suspicious activity in examination with openpose. *Unknown Journal*, 2023.
- [7] –. Intelligent offline exam monitoring system (iems) for identifying suspicious behavior. *International Journal of Computer Applications*, 2025.

Appendix A

Appendices

A.1 Appendix A

A.1.1 Use Case Diagram example (Online Shopping System)

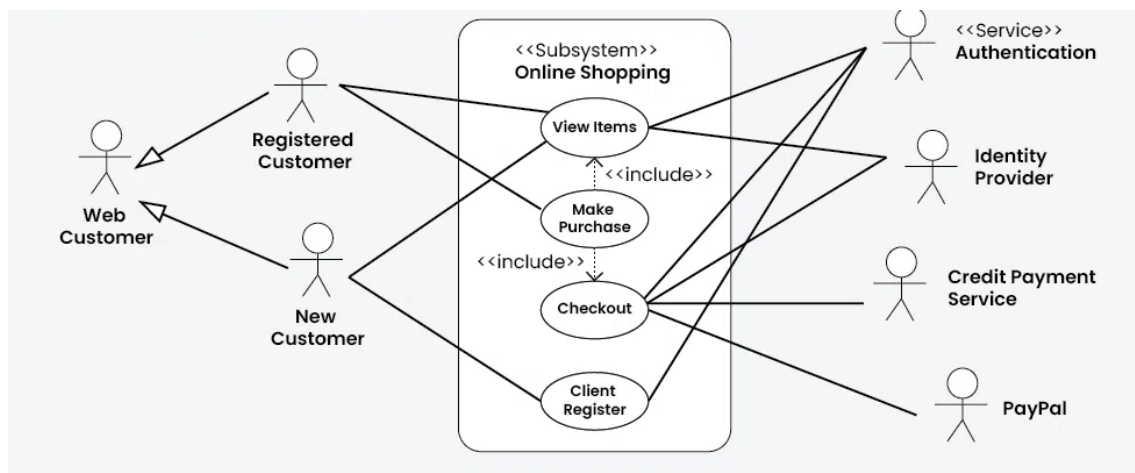


Figure A.1: Use Case Diagram for the Online Shopping System

A.1.2 Detail Use Case Example

ID	#1
Name	Overview elements
Short Description	All elements are shown in a list, where the user can set different filters.
Goal	Displaying elements in a changeable view.
Preconditions	The user got access to the system and is logged in. The user got the right to see schedules.
Success End Condition	The correct elements (filter and sorting) are displayed.
Fall End Condition	Elements not matching the criteria are displayed.
Stakeholder	Customer manager
Trigger	Login in as customer manager (because overview is on the starting screen for this role).
Normal Flow	1. The list of all elements matching default criteria are shown. 2. The user changes the filter criteria. 3. The user presses the Filter button. 4. The list shows all matching elements. Optional: 5. The user presses the Clear button. 6. The list of all elements matching default criteria are shown.
Alternative Flows	With click on the column headers, the list sorting can be changed. Different sorting for the different columns is described in the table below.
Includes	Login
Frequency of Use	About 50 times per day
Constraints and Special Requirements	None
Assumptions	None
Notes and Issues	None

Figure A.2: Detail Use Case Example

A.1.3 Event-Response Table for a Highway Intersection

Event	System State	Response
Road sensor detects vehicle entering left-turn lane.	Left-turn signal is red. Cross-traffic signal is green.	Start green-to-amber countdown timer for cross-traffic signal.
Green-to-amber countdown timer reaches zero.	Cross-traffic signal is green.	1. Turn cross-traffic signal amber. 2. Start amber-to-red countdown timer.
Amber-to-red countdown timer reaches zero.	Cross-traffic signal is amber.	1. Turn cross-traffic signal red. 2. Wait 1 second. 3. Turn left-turn signal green. 4. Start left-turn-signal countdown timer.
Pedestrian presses a specific walk-request button.	Pedestrian sign is solid Don't Walk. Walk-request countdown timer is not activated.	Start walk-request countdown timer.
Pedestrian presses walk-request button.	Pedestrian sign is solid Don't Walk. Walk-request countdown timer is activated.	Do nothing.
Walk-request countdown timer reaches zero plus the amber display time.	Pedestrian sign is solid Don't Walk.	Change all green traffic signals to amber.
Walk-request countdown timer reaches zero.	Pedestrian sign is solid Don't Walk.	1. Change all amber traffic signals to red. 2. Wait 1 second. 3. Set pedestrian sign to Walk. 4. Start don't-walk countdown timer.

Figure A.3: Example of Event Response Table

A.1.4 Story Board Example For Android App

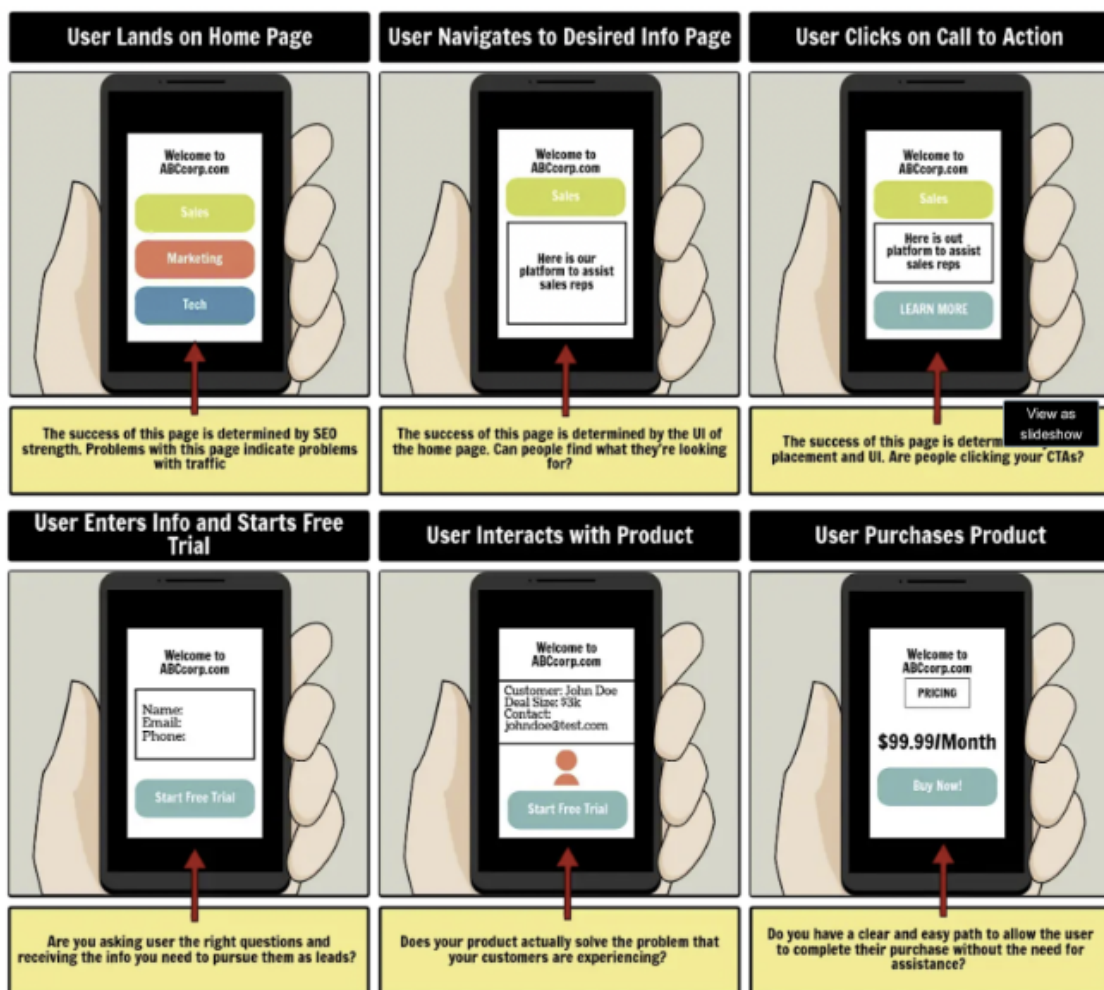


Figure A.4: Example of Story Board

A.2 Appendix B

A.2.1 Domain Model Example For Online Shopping

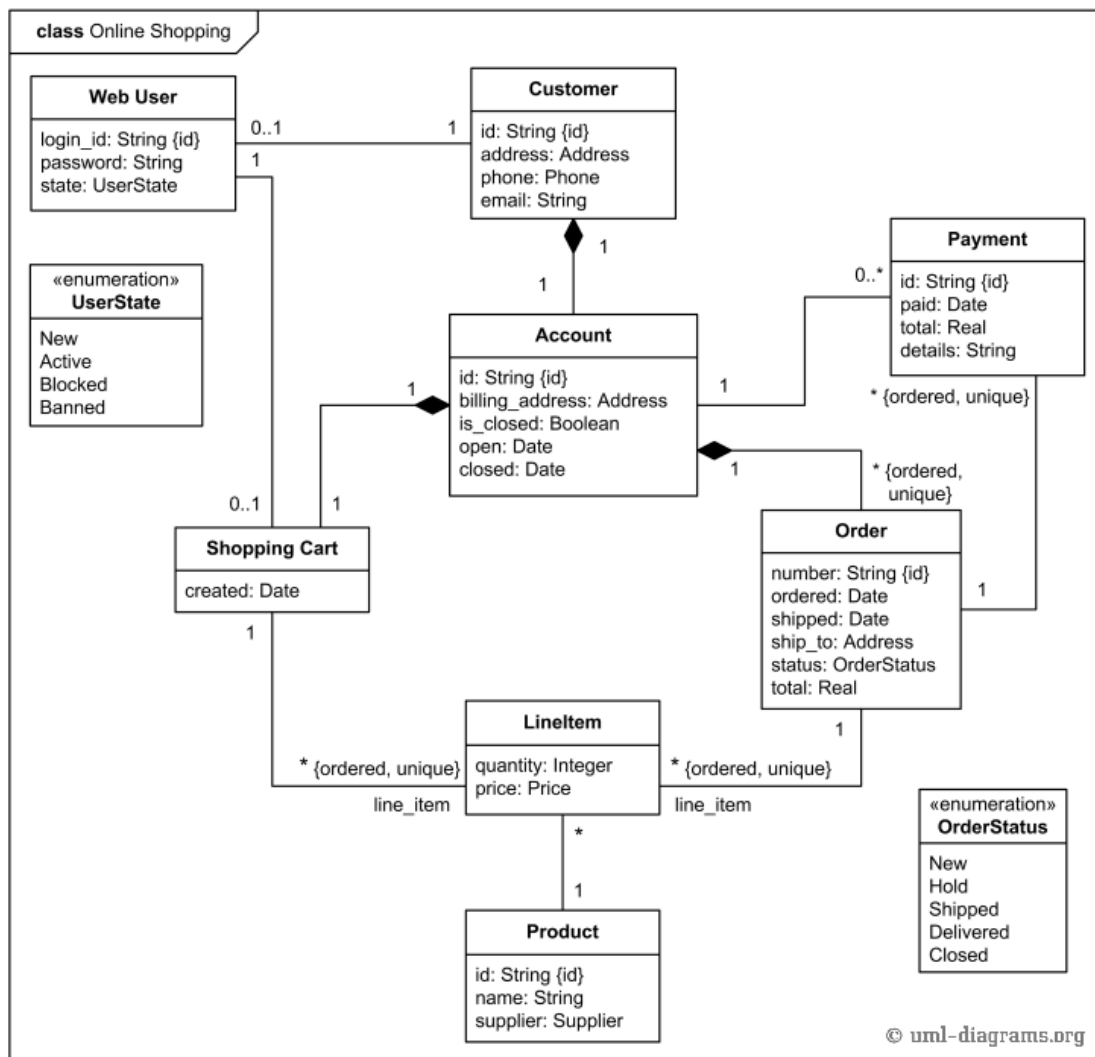


Figure A.5: Domain Model Example For Online Shopping Application

A.3 Appendix C

A.3.1 Box And Line Example For Online Shopping

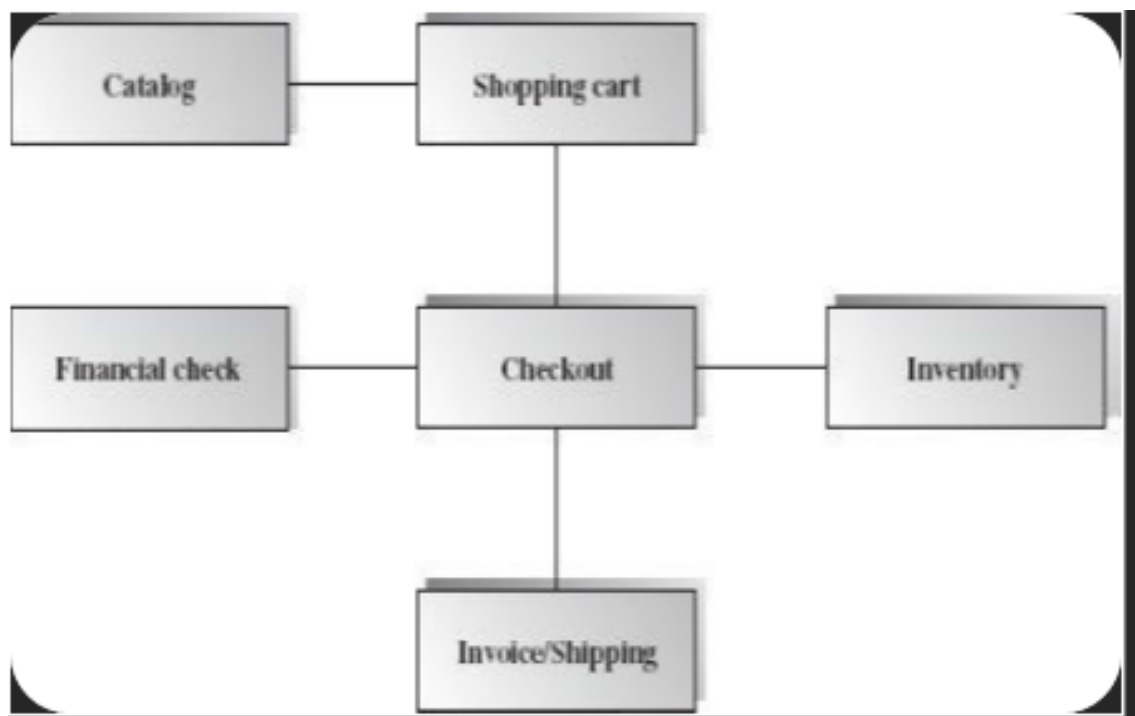


Figure A.6: Box and Line Diagram For Online Shopping Application

A.3.2 Architecture Pattern Example For Online Shopping

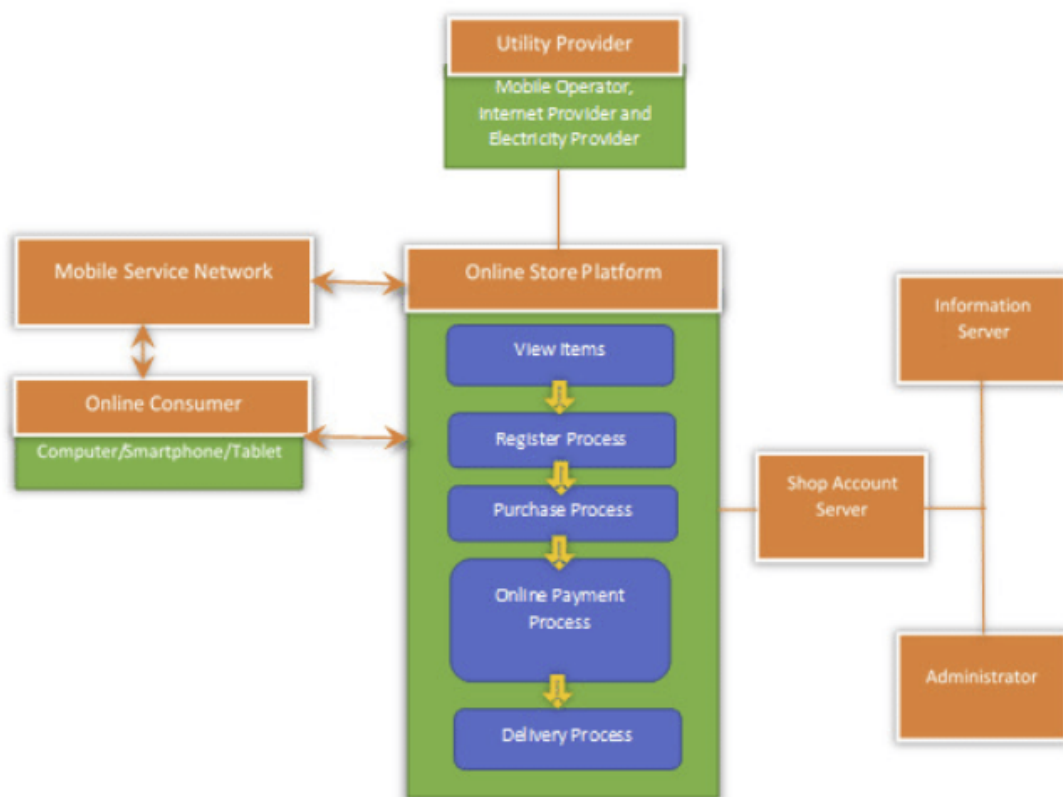


Figure A.7: Architecture Pattern For Online Shopping Application

A.4 Appendix D

A.4.1 Activity Diagram

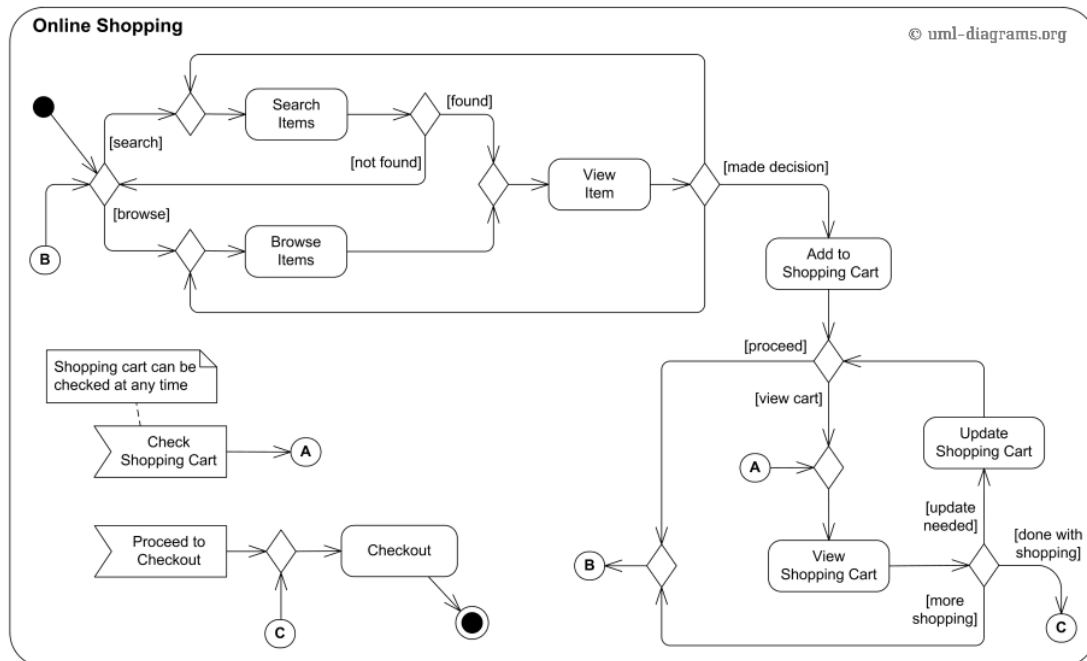


Figure A.8: Activity Diagram For Online Shopping Application

A.4.2 Class Diagram

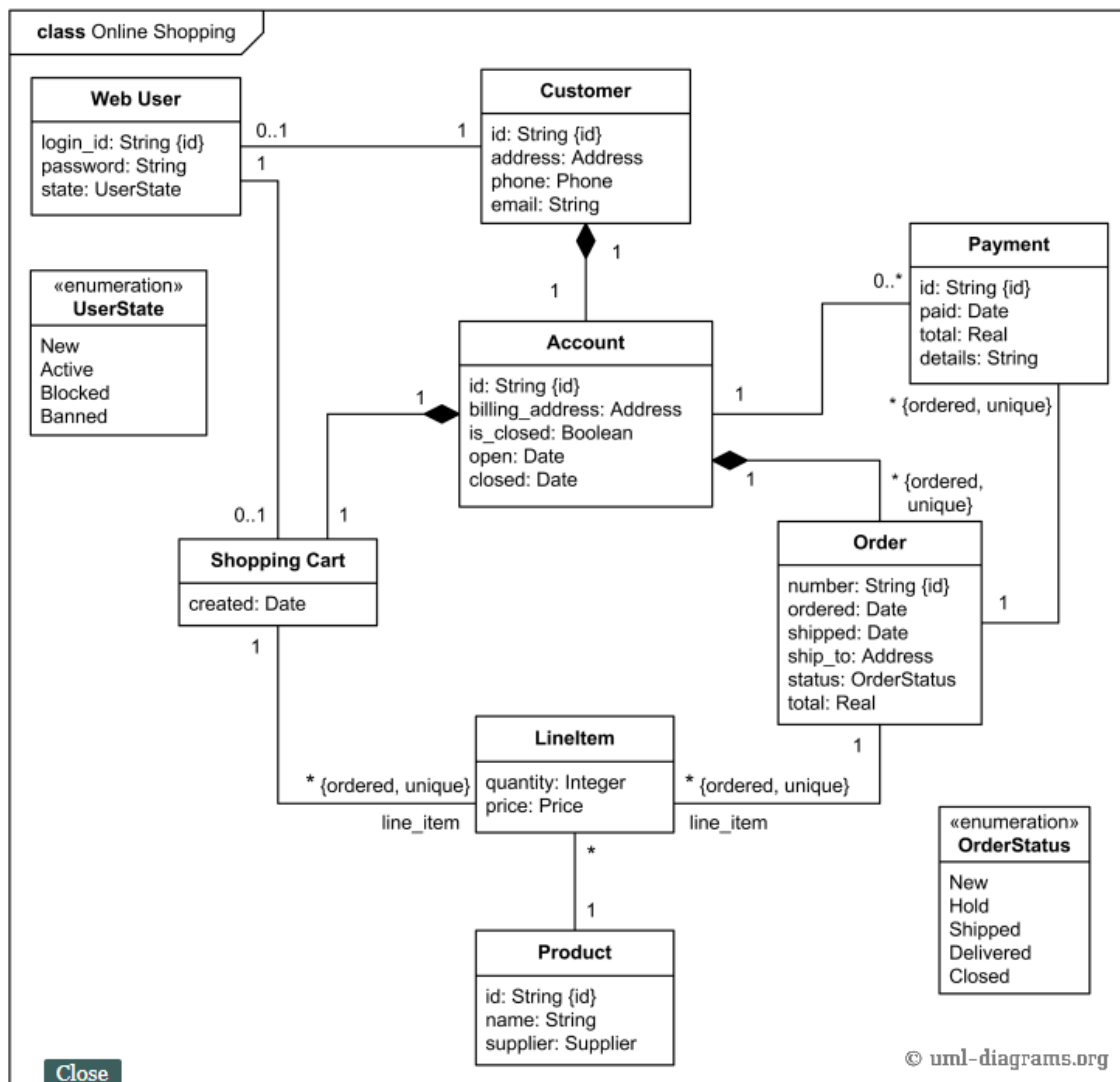


Figure A.9: Class Diagram For Online Shopping Application

A.4.3 Sequence Diagram

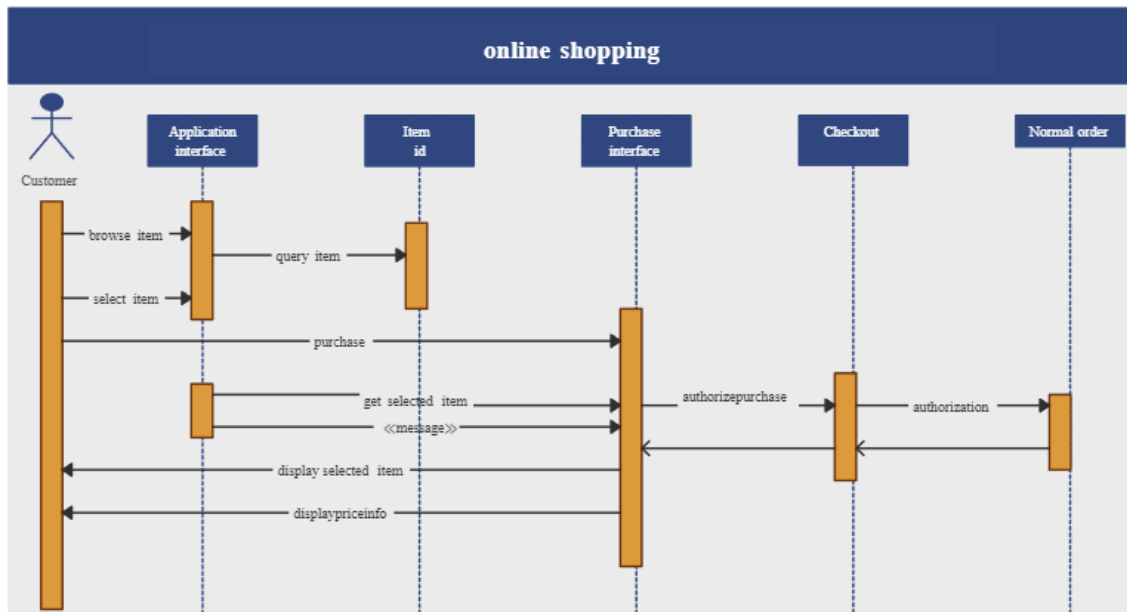


Figure A.10: Sequence Diagram For Online Shopping Application

A.4.4 State Transition Diagram

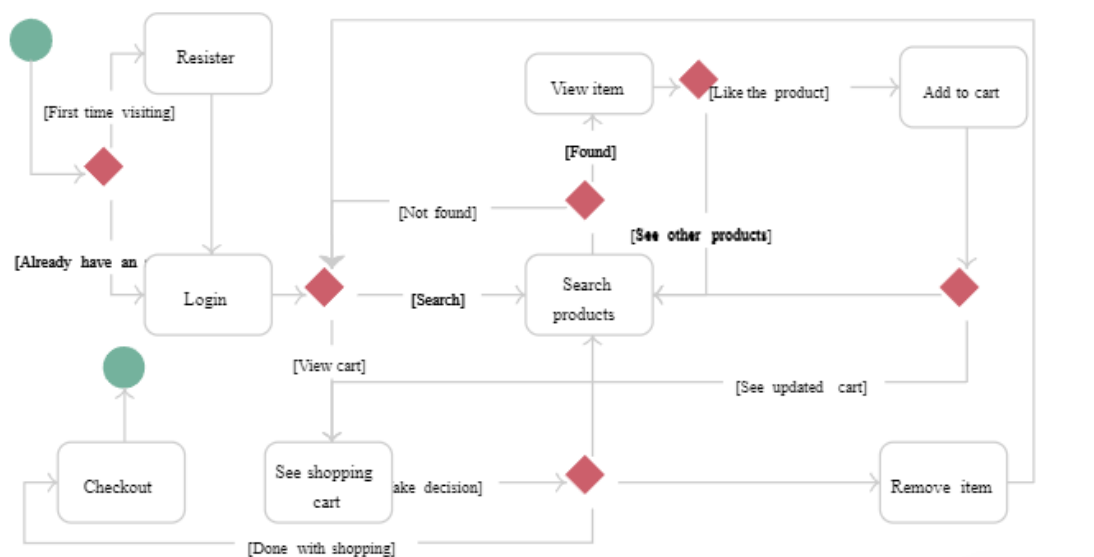


Figure A.11: State Transition Diagram For Online Shopping Application

A.4.5 Data Flow Diagram

Data Flow Diagram

Data flow diagram symbols, symbol names, and examples

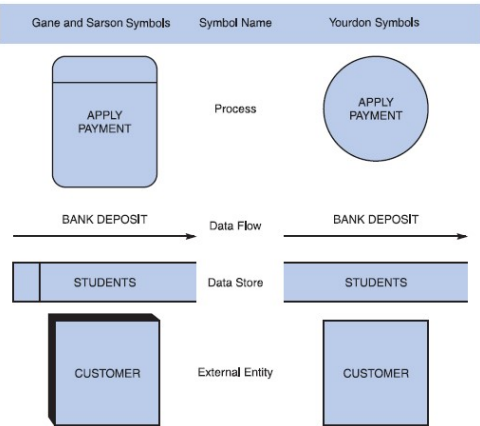


Figure B-5 Data flow diagram symbols, symbol names, and examples of the Gane and Sarson and Yourdon symbol sets.

Guidelines for Drawing DFDs

Step 1: Draw a Context Diagram: The first step in constructing a set of DFDs is to draw a context diagram. A **context diagram** is a top-level view of an information system that shows the system’s boundaries and scope. Data stores are not shown in the context diagram because they are contained within the system and remain hidden until more detailed diagrams are created.

Example

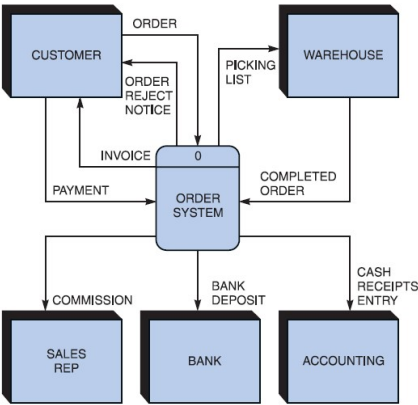


Figure B-6 Context diagram DFD for an order system.

Step 2: Draw a Diagram 0 DFD: To show the detail inside the black box, you create DFD diagram 0. **Diagram 0** zooms in on the system and shows major internal processes, data flows, and data stores. Diagram 0 also repeats the entities and data flows that appear in the context diagram. When you expand the context diagram into DFD diagram 0, you must retain all the connections that flow into and out of process 0.

Example

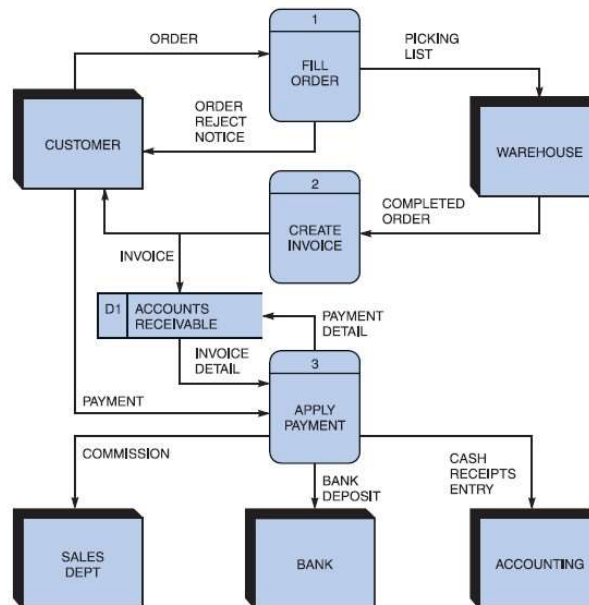


Figure B-7 Diagram 0 DFD for the order system.

Step 3: Draw the Lower-Level Diagrams:

To create lower-level diagrams, you must use leveling and balancing techniques. **Leveling** is the process of drawing a series of increasingly detailed diagrams, until all functional primitives are identified.

Leveling Example

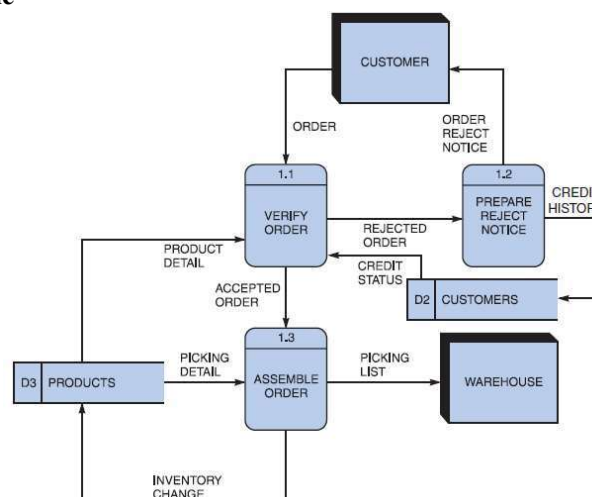


Figure B-8 Diagram 1 DFD shows details of the FILL ORDER process in the order system.